

DTMP-Prime: Reconstrução, Análise e Validação

Mateus de Jesus Mendes*

12 de agosto de 2026

Resumo

O presente projeto destina-se à reconstrução fiel e à validação do modelo *DTMP-Prime*, um preditor de eficiência de *Prime Editing* (PE) baseado na arquitetura Transformer, originalmente proposto por Alipanahi, Safari e Khanteymoori (*Molecular Therapy Nucleic Acids*, 2024). O trabalho foi conduzido a partir de um repositório público incompleto, com o duplo objetivo de (i) produzir uma implementação robusta, modular e utilizável em produção, e (ii) reproduzir integralmente a arquitetura descrita no artigo, por completude metodológica. A metodologia foi organizada em fases incrementais e individualmente validáveis, aferindo-se a contribuição isolada de cada componente arquitetural. Os resultados demonstram que a implementação atinge desempenho comparável ou levemente superior ao artigo na mesma sob as mesmas métricas (coeficiente de Spearman de 0,78 contra 0,77 reportado), e que, entre as camadas arquiteturais propostas, apenas as *features* escalares agregam sinal preditivo substancial, enquanto o *encoding* e o *embedding* oriundo do DNABERT, não contribuem significativamente para a tarefa de predição de eficiência *on-target*. Adicionalmente, uma análise de validação utilizando-se GroupKFold por alvo revela que a métrica de generalização real ($r = 0,67$) é substancialmente inferior à métrica com vazamento estrutural ($r = 0,89$), constituindo uma avaliação mais rigorosa que a original. Por fim, a validação externa contra dados independentes — a varredura sistemática de comprimentos de PBS e RTT conduzida por Sousa *et al.* (*Nature Biomedical Engineering*, 2024) para a correção da mutação *CFTR* F508del — delimita o domínio de aplicabilidade do modelo: a ordenação predita reproduz perfeitamente o ramo ascendente do perfil experimental de PBS dentro da faixa de treino, mas não identifica o ótimo e não apresenta correlação com a medida experimental sobre a grade completa, cujo eixo de RTT situa-se inteiramente fora do domínio de treino. O repositório resultante encontra-se disponível em: <https://github.com/mateusjmd/DTMP-Prime>.

Palavras-chave: *Prime Editing*, pegRNA, Transformer, predição de eficiência, DNABERT.

*mateus25032@ilum.cnpem.br

Sumário

1	Introdução	5
1.1	Contextualização	5
1.2	Motivação	5
1.3	Objetivos	6
1.4	Visão geral da metodologia	6
1.5	Modelos de produção e completude	7
1.6	Organização do documento	7
2	Metodologia	7
2.1	Estrutura geral e ordem das fases	7
2.2	Correção de defeitos estruturais do repositório	8
2.2.1	Artefatos e módulos ausentes	8
2.2.2	Falhas de robustez	9
2.2.3	Inconsistências lógicas entre componentes	9
2.3	Fase 1 — Reconstrução do <i>wide target</i> de 47 pb	10
2.3.1	Objetivos e motivação	10
2.3.2	Fundamentação teórica e relação com o artigo	10
2.3.3	Modificações por arquivo	10
2.3.4	Decisões de projeto e desafios	12
2.3.5	Limitação de reprodutibilidade	12
2.3.6	Fluxo de execução e validação	12
2.3.7	Impacto sobre as fases subsequentes	12
2.4	Fase 2 — Integração das <i>features</i> escalares	13
2.4.1	Objetivos e motivação	13
2.4.2	Fundamentação teórica e relação com o artigo	13
2.4.3	Modificações por arquivo	13
2.4.4	Reconstrução das <i>features</i> em inferência	15
2.4.5	Tecnologias utilizadas	16
2.4.6	Decisões de projeto e desafios	16
2.4.7	Limitações e impacto	16
2.5	Fase 3 — Reconstrução do <i>encoding</i> $8 \times L$	16
2.5.1	Objetivos, motivação e fundamentação	16
2.5.2	Fundamentação teórica e relação com o artigo	17
2.5.3	Modificações por arquivo	17
2.5.4	Fluxo de execução, tecnologias e decisões de projeto	19
2.5.5	Limitações, impacto e análise crítica	20
2.6	Fase 4 — Camada de <i>embedding</i> DNABERT	20

2.6.1	Objetivos, motivação e fundamentação	20
2.6.2	Viabilização e desafios de infraestrutura	20
2.6.3	Modificações por arquivo	21
2.6.4	Duas abordagens e decisões de projeto	23
2.6.5	Sequência processada e fluxo	23
2.6.6	Limitações e impacto	23
2.7	Fase 5 — Protocolos de validação	24
2.7.1	Objetivos e motivação	24
2.7.2	Fundamentação teórica e relação com o artigo	24
2.7.3	Modificações por arquivo	24
2.7.4	Fluxo de execução	26
2.7.5	Limitações e impacto	26
2.8	Fase 0 — Pré-treino em ClinVar	27
2.8.1	Objetivos e motivação	27
2.8.2	Fundamentação teórica e relação com o artigo	27
2.8.3	Análise de viabilidade	27
2.8.4	Proposta de versão factível	28
2.8.5	Impacto e considerações finais	29
2.9	Interface Gráfica	29
2.9.1	Arquitetura	29
2.9.2	Contribuições em relação ao trabalho original	29
2.9.3	Tutorial	30
3	Resultados e Discussão	32
3.1	Quadro consolidado das medições por fase	32
3.2	Resultado central	32
3.3	Validação: três protocolos e a questão do vazamento	34
3.4	Sobre a legitimidade dos métodos de validação cruzada	34
3.5	Comparação com o artigo e impacto das adaptações	36
3.6	<i>Trade-offs</i> e fontes de erro	36
3.7	Infraestrutura, custo computacional e <i>bottlenecks</i>	36
3.7.1	<i>Hardware</i>	37
3.7.2	Tempo de computação	37
3.7.3	Memória	37
3.7.4	Pré-cálculo como estratégia de mitigação	38
3.7.5	Dependências de <i>software</i>	38
3.7.6	Síntese dos pontos de <i>bottlenecking</i>	38

4	Validação Externa	39
4.1	Objetivo e escopo	39
4.2	Mapeamento do <i>design</i> experimental para o <i>pipeline</i>	40
4.3	Divergências metodológicas	41
4.4	Protocolo implementado	42
4.5	Resultados	45
4.6	Limitações e alcance da conclusão	49
4.7	Implicações para o trabalho	50
5	Conclusão	52
5.1	Síntese dos objetivos alcançados	52
5.2	Principais contribuições	52
5.3	Limitações da implementação	53
5.4	Trabalhos futuros e possíveis extensões	54
5.5	Considerações finais	54
6	Uso de Inteligência Artificial Generativa	55
7	Agradecimentos	55
	Referências	56

1 Introdução

1.1 Contextualização

O *Prime Editing* (PE) [2] constitui uma tecnologia de edição genômica de precisão que expande substancialmente as capacidades do sistema CRISPR-Cas9, permitindo a introdução dirigida de substituições, inserções e deleções sem a necessidade de quebras de fita dupla ou de moldes doadores. A eficiência de uma edição por PE depende criticamente do desenho do *prime editing guide RNA* (pegRNA) — em particular de seus componentes *Primer Binding Site* (PBS) e *Reverse Transcriptase Template* (RTT) — bem como do contexto genômico do sítio-alvo. Como a eficiência experimental de um dado pegRNA não é trivialmente previsível a partir de sua sequência, modelos computacionais de predição tornam-se instrumentos essenciais para o desenho racional de experimentos de PE, reduzindo o custo de triagem empírica de candidatos.

O modelo DTMP-Prime [1], proposto por Alipanahi, Safari e Khanteymooori (*Molecular Therapy Nucleic Acids*, 2024), aborda esse problema por meio de uma arquitetura baseada em Transformers [12] com mecanismo de *multi-head attention*, integrando três componentes principais: um esquema de codificação par-a-par PegRNA-DNA de dimensão $8 \times L$, uma camada de *embedding* baseada em DNABERT [5], e um conjunto de *features* manuais (*hand-crafted features*) associados a propriedades termodinâmicas e estruturais das sequências.

1.2 Motivação

A reprodução independente de modelos de aprendizado profundo publicados é um pilar da ciência computacional rigorosa, tanto para validação de resultados quanto para viabilização de extensões metodológicas. No caso do DTMP-Prime, o repositório público disponível¹ encontrava-se incompleto (desprovido de artefatos de treinamento, de módulos referenciados no código e de arquivos de modelo) e continha inconsistências entre a descrição textual do artigo e a implementação distribuída. Tal cenário estabelece a motivação central do presente projeto: reconstruir fielmente o modelo a partir do material disponível, identificando e documentando cada divergência, adaptação e limitação de reprodutibilidade encontrada.

¹Repositório original disponível em: **DTMP-Prime**

1.3 Objetivos

Objetivo geral

Reconstruir, validar e analisar o modelo DTMP-Prime, produzindo tanto uma implementação utilizável em produção quanto uma reprodução metodologicamente completa da arquitetura descrita no artigo de referência.

Objetivos específicos

- Corrigir os defeitos estruturais do repositório que impediam a execução correta do *pipeline* de inferência e de treinamento.
- Reconstruir cada componente arquitetural do modelo em fases incrementais e individualmente validáveis.
- Quantificar, de forma isolada, a contribuição de cada camada arquitetural para o desempenho preditivo.
- Estabelecer um protocolo de validação rigoroso, capaz de distinguir interpolação intra-alvo de generalização real a novos *loci* genômicos.
- Documentar exaustivamente as divergências entre a descrição do artigo e a implementação, bem como os limites de reprodutibilidade impostos pelo material disponível.
- Desenvolver uma validação das previsões estruturais e de eficiência da edição geradas pelo modelo de produção, comparando-as com resultados previamente obtidos a partir de estudos existentes em literatura.

1.4 Visão geral da metodologia

A metodologia adotada organiza-se em torno do princípio de decomposição incremental validável, isto é, cada componente do modelo é implementado, integrados e avaliado isoladamente, sobre um *baseline* bem definido, antes que o componente seguinte seja incorporado. Essa abordagem permite atribuir de forma inequívoca o efeito de cada camada arquitetural, evitando a confusão de variáveis que ocorreria caso múltiplos componentes fossem adicionados simultaneamente.

O trabalho foi estruturado em fases numeradas, de forma que a ordem de *desenvolvimento* das fases não coincide necessariamente com a ordem de execução no *pipeline* final, uma distinção detalhada ao longo do documento. Cada fase foi submetida a validação empírica contra dados de referência — seja o conjunto de teste com eficiência medida experimentalmente, seja os resultados de referência extraídos de figuras e tabelas suplementares do artigo, seja comparando ao fim com dados experimentais de outros estudos.

1.5 Modelos de produção e completude

Todas as decisões metodológicas do projeto foram orientadas por duas frentes complementares, cujo equilíbrio permeia a totalidade do trabalho.

Frente de Produção

Visa a construção de uma implementação robusta, modular, reutilizável, reproduzível e eficiente, adequada ao uso prático. Nessa frente, privilegiaram-se a correção de defeitos, a consistência entre treino e inferência, a organização do código em módulos coesos e a seleção do modelo de melhor desempenho para operação efetiva no contexto de rotina laboratorial.

Frente de Completude

Visa a reprodução integral da metodologia do artigo, implementando todos os componentes arquiteturais relevantes, sem objetivar a maximização de desempenho do modelo. Nessa frente, a implementação frequentemente ultrapassou o mínimo necessário à execução, a fim de documentar cada etapa desenvolvida e estabelecer-se enquanto referência para pesquisas futuras.

Ao longo do documento, explicita-se, para cada decisão relevante, se esta priorizou a produção, a completude, ou um equilíbrio entre ambas, discutindo-se os *trade-offs* decorrentes.

1.6 Organização do documento

O restante do documento organiza-se como segue. A Seção 2 descreve detalhadamente a metodologia, apresentando cada fase do projeto em subseção própria, na ordem cronológica de execução. A Seção 3 apresenta e discute os resultados obtidos, comparando a implementação desenvolvida com a proposta do artigo e analisando os *trade-offs* assumidos. A Seção 4 desenvolve a validação do modelo de produção a partir da variação de diferentes comprimentos de PBS e RTT para a correção da F508del no gene CFTR, orientando-se a partir dos resultados de Sousa *et al.* [11]. A Seção 5 sintetiza as contribuições, as limitações e os principais aprendizados, apontando trabalhos futuros e possíveis extensões.

2 Metodologia

2.1 Estrutura geral e ordem das fases

O projeto foi decomposto em seis fases. Essa distinção conceitual, mantida ao longo de todo o trabalho, separa a ordem de desenvolvimento das fases da ordem de execução no

pipeline final. Cada uma das fases foram desenvolvidas na sequência que maximiza a validabilidade incremental, isto é, na ordem em que cada uma pode ser verificada sobre uma base já consolidada, e não necessariamente na ordem em que os dados fluem durante a execução.

Tabela 1: Fases do projeto: ordem de desenvolvimento e papel no *pipeline*.

Fase	Componente	Frente predominante	Situação
1	<i>wide target</i> de 47 pb	Produção	Concluída
2	14 <i>features</i> escalares	Produção + Completude	Concluída
3	<i>Encoding</i> $8 \times L$	Completude	Concluída
4	<i>Embedding</i> DNABERT	Completude	Concluída
5	Validação (três protocolos)	Produção + Completude	Concluída
0	Pré-treino ClinVar	Completude	Irreproduzível

Antes da descrição das fases, registram-se os defeitos estruturais do repositório original que foram corrigidos como pré-requisito para qualquer execução confiável.

2.2 Correção de defeitos estruturais do repositório

O repositório distribuído apresentava um conjunto de defeitos estruturais que impediam tanto a execução do *pipeline* de inferência quanto o treinamento dos modelos. A identificação e a correção desses defeitos constituíram a etapa preliminar do trabalho, inserindo-se predominantemente na frente de produção. Os problemas podem ser agrupados em três categorias: artefatos ausentes, falhas de robustez, e inconsistências lógicas entre componentes.

2.2.1 Artefatos e módulos ausentes

Parte substancial do material necessário à execução não acompanhava o repositório, exigindo reconstrução ou remapeamento:

- **Módulo de predição inexistente:** O código-fonte referenciava um pacote de rotinas de predição que não constava do repositório. Constatou-se que suas funcionalidades (carregamento de dados e avaliação) haviam sido redistribuídas em módulos de nível superior, exigindo o remapeamento das importações para os arquivos corretos.
- **Artefatos de modelo e de treinamento não distribuídos:** Os arquivos de pesos dos modelos treinados, bem como os artefatos gerados durante o treinamento, encontravam-se ausentes, o que inviabilizava a inferência direta e tornava obrigatória a reconstrução completa do processo de treino.
- **Recursos de *front-end* e de validação ausentes:** Diversos arquivos auxiliares (de interface, de validação de fases e de treinamento em formato de *Jupyter Notebook*) não

estavam diretamente presentes no repositório, tendo sido reconstruídos ao longo do projeto conforme a necessidade.

- **Refatoração e renomeação de arquivos:** Diversos módulos e *scripts* presentes no repositório se encontravam nomeados utilizando caracteres, como espaços, o que dificultava a chamada interna dos mesmos ao longo do desenvolvimento e correções em outros *scripts* dependentes destes. Dessa forma, parte dos arquivos originais foram renomeados, facilitando o seu uso e legibilidade geral ao longo do projeto e a partir de arquivos dependentes.

2.2.2 Falhas de robustez

A categoria mais insidiosa de defeitos não produzia mensagens de erro explícitas, manifestando-se apenas como comportamento incorreto, o que dificultava significativamente o diagnóstico:

- **Supressão indevida de exceções:** No módulo principal de execução, blocos de captura genérica de exceções ocultavam falhas de importação e de processamento, convertendo erros fatais em falhas silenciosas. A divergência de comportamento entre os modos de depuração ativado e desativado constituiu, ela própria, o sinal diagnóstico que revelou o problema.
- **Estruturação condicional incorreta:** No módulo de mutação do alvo, atribuições essenciais ao fluxo encontravam-se indevidamente aninhadas em blocos condicionais de depuração, de modo que deixavam de ser executadas em operação normal, comprometendo o resultado sem gerar erro aparente.

2.2.3 Inconsistências lógicas entre componentes

A terceira categoria compreende incoerências entre a especificação e a implementação, ou entre componentes que deveriam operar de forma alinhada:

- **Desalinhamento entre representação de treino e de inferência:** A rotina de carregamento de dados empregava, como sequência-alvo, uma representação distinta daquela especificada pela estrutura de referência, estabelecendo uma incoerência entre o que o modelo observava em treino e o que recebia em inferência.
- **Ausência de geração dos melhores candidatos:** A rotina de predição não construía a estrutura de saída com os candidatos mais bem classificados, fazendo o *pipeline* retornar resultado vazio ainda quando o processamento ocorria corretamente.
- **Invocação redundante da camada de decisão:** Na função de execução *forward* do modelo, a camada integralmente conectada era invocada antes da agregação completa

dos vetores de entrada, o que, ao integrar ramos arquiteturais adicionais, produzia incompatibilidade dimensional.

2.3 Fase 1 — Reconstrução do *wide target* de 47 pb

2.3.1 Objetivos e motivação

A Fase 1 tem por objetivo garantir que a sequência-alvo fornecida ao modelo, tanto em treino quanto em inferência, corresponda à representação genômica correta de 47 pb, e não ao *spacer* de 20 pb. Constitui o alicerce de todas as fases subsequentes, de modo que qualquer desalinhamento entre a representação de treino e a de inferência propagasse silenciosamente por todo o *pipeline*, comprometendo a validade de todas as medições posteriores. Essa fase insere-se predominantemente na frente de produção, por assegurar a consistência operacional do modelo sobre sequências novas.

2.3.2 Fundamentação teórica e relação com o artigo

O artigo, por meio da Tabela S5, especifica a estrutura da *wide target sequence* como a concatenação determinística de quatro segmentos:

$$\underbrace{[4 \text{ pb}]}_{\text{vizinhança } 5'} \quad \underbrace{[20 \text{ pb}]}_{\text{protospacer}} \quad \underbrace{[3 \text{ pb}]}_{\text{PAM (xGG)}} \quad \underbrace{[20 \text{ pb}]}_{\text{vizinhança } 3'} = 47 \text{ pb} \quad (1)$$

A verificação empírica confirmou essa estrutura em 100% dos registros da Tabela S5, com o *protospacer* ocupando invariavelmente as posições [4:24]. O *guide* do conjunto de dados apresenta um nucleotídeo G artificial em sua primeira posição (introduzido para a transcrição pelo promotor U6), enquanto o *protospacer* emprega o nucleotídeo genômico real. A distinção entre essas duas representações (o *guide* de 20 pb e o *wide target* de 47 pb) é a origem do defeito corrigido nesta fase.

2.3.3 Modificações por arquivo

A reconstrução do *wide target sequence* envolveu a modificação de arquivos existentes e a criação de arquivos novos. A Tabela 2 sintetiza as intervenções, discutidas em detalhe a seguir.

Correção na função de carregamento (`read_data.py`)

A raiz do desalinhamento localizava-se na função de leitura e tokenização do conjunto de dados. Em sua versão original, essa função utilizava a coluna correspondente ao *guide* de 20 pb como sequência-alvo do modelo que, com a correção, passou a empregar a coluna do *wide target* de 47 pb. À rotina de montagem de cada registro adicionou-se ainda

Tabela 2: Intervenções por arquivo na Fase 1.

Arquivo	Natureza	Intervenção
<code>read_data.py</code>	Modificado	Correção da sequência-alvo na função de carregamento, de <i>guide</i> (20 pb) para <i>wide target</i> (47 pb), com verificação de comprimento.
<code>wide_target.py</code>	Novo	Função de extração do <i>wide target</i> de 47 pb a partir do contexto genômico, para uso em inferência.
<code>target_mutation.py</code>	Modificado	Integração da extração do <i>wide target</i> ao caminho de inferência, com calibração de deslocamentos por sentido de fita.
<code>validar_fase1.py</code>	Novo	Rotina de validação da estrutura reconstruída contra os critérios de posição do <i>protospacer</i> e de terminação do PAM (<i>Protospacer Adjacent Motif</i>).

uma verificação de comprimento, que assegura invariavelmente a extensão de 47 pb da sequência-alvo antes de sua tokenização.

Extração do *wide target* em inferência (`wide_target.py`)

Em inferência, o *wide target* não é derivável do *spacer*, devendo ser recortado do contexto genômico. Para tanto, criou-se um módulo dedicado, que expõe uma função de extração recebendo a sequência genômica, a posição de corte e o sentido da fita, e retornando a janela de 47 pb correspondente. A separação dessa lógica em módulo próprio atende à frente de produção, ao promover a modularidade e a reutilização do procedimento de extração.

Integração ao caminho de inferência (`target_mutation.py`)

A rotina de predição foi modificada para invocar a extração do *wide target* para cada candidato, substituindo a representação anterior. Os deslocamentos que localizam a janela de 47 pb foram calibrados empiricamente contra o identificador do *spacer* e o PAM, exigindo tratamento distinto para cada sentido de fita:

- **Fita positiva:** `proto_start = cut - 17; wide = target_fa[proto_start - 4 : proto_start + 43]`.
- **Fita negativa:** `proto_start = cut - 4; wide = revcomp(target_fa[proto_start - 23 : proto_start + 24])`.

É necessário ressaltar que é adotada a convenção de posição de corte indexada a partir de zero, situada três pares de base antes do PAM.

Rotina de validação (`validar_fase1.py`)

Para atestar a correção da reconstrução, criou-se uma rotina de validação independente, que verifica, para cada candidato, dois critérios simultâneos: a recaída do *protospacer* extraído sobre as posições [4:24] e a terminação do PAM em GG.

2.3.4 Decisões de projeto e desafios

O principal desafio da fase residuiu na determinação da convenção exata da posição de corte, resolvida pelo procedimento de calibração descrito, que testou múltiplos deslocamentos contra dois critérios independentes: a correspondência do *protospacer* e a terminação do PAM. A fita negativa revelou-se particularmente sensível a erros de deslocamento unitário, tendo sido validada de forma isolada.

2.3.5 Limitação de reprodutibilidade

A convenção de deslocamento da fita negativa ($\text{cut} - 4$) difere, por uma unidade, do espelhamento exato da fita positiva. Essa assimetria reflete a convenção interna do *pipeline* quanto à posição precisa do corte, e foi determinada empiricamente, não a partir da especificação do artigo, constituindo, uma decisão de projeto documentada, e não uma reprodução literal do método original.

2.3.6 Fluxo de execução e validação

O fluxo consolidado da fase compreende, em treino, a leitura da coluna do *wide target* com verificação de comprimento (`read_data.py`) e, em inferência, a extração da janela de 47pb a partir do genoma (`wide_target.py`), integrada à rotina de predição (`target_mutation.py`). O procedimento de validação (`validar_fase1.py`) foi concluído com nenhuma falha em ambos os sentidos de fita, confirmando a consistência entre a representação de treino e a de inferência.

2.3.7 Impacto sobre as fases subsequentes

A correção do alinhamento constitui pré-requisito de todas as fases seguintes. Sua ausência havia produzido uma divergência aparente entre o modelo Transformer e o modelo de referência baseado em XGBoost [3], posteriormente reinterpretada, à luz da Fase 5, como manifestação de que o Transformer capturava sinal de sequência não acessível ao critério de comparação empregado, e não como regressão de desempenho. Ao consolidar a consistência entre treino e inferência, a Fase 1 viabiliza a atribuição inequívoca do efeito de cada componente arquitetural introduzido nas fases posteriores.

2.4 Fase 2 — Integração das *features* escalares

2.4.1 Objetivos e motivação

A Fase 2 incorpora ao Transformer um conjunto de *features* escalares associadas a propriedades termodinâmicas e estruturais das sequências. Sua motivação decorre diretamente da análise de importância de *features* apresentada no artigo, que identifica a atividade de Cas9 (*DeepSpCas9*), o enovelamento de RNA (*RNA folding*) e o conteúdo GC do PBS como os preditores de maior peso — dos quais todos são escalares, e nenhum diretamente acessível à representação puramente sequencial da Fase 1. Essa fase atende simultaneamente às duas frentes: à produção, por constituir o modelo de melhor desempenho e viabilizar sua operação em inferência; e à completude, por reproduzir fielmente a camada de *features* manuais descrita no artigo.

2.4.2 Fundamentação teórica e relação com o artigo

O artigo integra *features* manuais à camada profunda do Transformer, concatenando-as à saída do mecanismo de atenção antes da camada de decisão. O conjunto de dados da Tabela S5 fornece dezessete *features* pré-computadas (colunas 8 a 24), abrangendo temperaturas de *melting* (T_m), contagens e frações de conteúdo GC, energias livres mínimas de enovelamento (*MFE*, do inglês *Minimum Free Energy*) e o *score DeepSpCas9*.

2.4.3 Modificações por arquivo

A integração das *features* escalares envolveu intervenções coordenadas em quatro arquivos existentes e a criação de um módulo novo para a reconstrução dos *features* em inferência. A Tabela 3 sintetiza as intervenções, cuja coordenação é essencial, pois o vetor de *features* precisa fluir de forma consistente pela arquitetura, pelo carregamento, pelo laço de treino e pela avaliação.

Tabela 3: Intervenções por arquivo na Fase 2.

Arquivo	Natureza	Intervenção
<code>train_models.py</code>	Modificado	Ativação da concatenação das <i>features</i> na classe do Transformer e passagem das <i>features</i> no laço de treino.
<code>read_data.py</code>	Modificado	Produção da coluna de <i>features</i> normalizadas na função de carregamento, com estatísticas derivadas apenas do treino.
<code>evaluate.py</code>	Modificado	Passagem das <i>features</i> à chamada do modelo na rotina de avaliação.
<code>features_inferencia.py</code>	Novo	Recálculo das 14 <i>features</i> em inferência e sua normalização com estatísticas do treino.

Ativação da concatenação na arquitetura (`Train_models.py`)

A classe `TransformerEncoderDecoderModelOrder3` já dimensionava a camada totalmente conectada para receber os *features*, por meio de um parâmetro de dimensão das *features*; entretanto, o método `forward` não os recebia nem os concatenava. A intervenção completou essa ligação em dois pontos: no método `__init__`, preservou-se o dimensionamento da camada de decisão para acomodar a dimensão adicional; no método `forward`, a assinatura passou a receber o vetor de *features*, que é concatenado à saída do mecanismo de atenção antes da camada de decisão. A rotina de treino da mesma unidade foi ajustada para construir o tensor de *features* a partir do lote e repassá-lo à chamada do modelo.

Cuidado de robustez: retrocompatibilidade

As referências à dimensão do vetor de *features* no método `forward` foram protegidas por acesso condicional, de modo que modelos serializados antes desta intervenção — desprovidos do parâmetro correspondente — sejam interpretados como possuidores de dimensão nula, sem gerar erro. Essa precaução, alinhada à frente de produção, viabiliza a coexistência de múltiplas versões de modelo sob um mesmo código de inferência.

Produção das *features* normalizadas no carregamento (`read_data.py`)

A função de carregamento e tokenização foi estendida para produzir, por registro, uma coluna adicional contendo o vetor de *features* já normalizadas. A normalização por *z-score* emprega estatísticas de média e desvio padrão calculadas exclusivamente sobre a partição de treino, armazenadas em disco para posterior reaplicação em inferência. Essa decisão previne o vazamento de informação do conjunto de teste para a normalização e assegura a consistência treino-inferência.

14 *features* vs. 17 *features*

A análise empírica revelou que a exclusão de três *features* da temperatura de *melting* ($Tm2$, $Tm3$ e ΔTm) melhora o desempenho do modelo, elevando o coeficiente de Pearson de 0,71 (17 *features*) para 0,74 (14 *features*). Tais *features*, classificadas como de baixa importância no artigo original, contribuíam com ruído em vez de sinal. A seleção final das *features* foi codificada, na função de carregamento, por meio de uma lista explícita de índices de coluna. Esta constitui uma adaptação em relação ao artigo, justificada empiricamente e alinhada à frente de produção, preservando-se a completude pela documentação do resultado com o conjunto integral de dezessete variáveis.

Passagem das *features* na avaliação (`evaluate.py`)

A rotina de avaliação, invocada a cada época para a seleção do melhor modelo, foi ajustada de forma simétrica ao laço de treino, passando-se a construir o tensor de *features* a partir do lote de teste e a repassá-lo à chamada do modelo. A simetria entre treino e avaliação é essencial, de modo que a ausência da passagem das *features* em qualquer dos

dois caminhos produziria incompatibilidade dimensional ou avaliação incorreta.

Recálculo das *features* em inferência (`features_inferencia.py`)

Durante o treino, as *features* provêm prontas da Tabela S5 enquanto que, durante a inferência, devem ser recalculadas para cada candidato. Criou-se, para tanto, um módulo dedicado, que expõe uma função de cálculo das catorze *features* a partir das sequências de PBS, RTT e *guide*, e do *score DeepSpCas9* pré-computado, bem como uma classe responsável por carregar as estatísticas de normalização do treino e aplicar a transformação de *z-score*. A separação dessa lógica em módulo próprio atende à frente de produção, ao encapsular o recálculo das *features* de forma reutilizável e consistente com o procedimento de treino.

2.4.4 Reconstrução das *features* em inferência

Assim como descrito anteriormente, durante o treino, as *features* provêm prontas da Tabela S5 e durante a inferência devem ser recalculadas para cada candidato, de modo que a reconstrução das *features* nessa etapa constitui o trabalho central desta fase, predominantemente pertencente à frente de produção, pois viabiliza a operação do modelo em sequências novas. As fórmulas foram determinadas por engenharia reversa e validadas célula a célula contra o gabarito da Tabela S5, com fidelidade média de 99,1% sobre cem amostras. A Tabela 4 resume as construções.

Tabela 4: Fórmulas de reconstrução das 14 *features* e fidelidade contra a Tabela S5.

<i>Feature</i>	Construção	Fidelidade
GC count 1–3	Contagem de G+C em PBS / RTT / PBS+RTT	100%
GC contents 1–3	Fração de G+C $\times 100$	100%
Tm1	$Tm_{NN}(\text{PBS}, R_DNA_NN1)$	100%
Tm4	$Tm_{NN}(\text{RTT}, R_DNA_NN1)$	100%
MFE_1	$mfe(\text{guide} + \text{scaffold} + rc(\text{ext}) + \text{polyT})$	99%
MFE_2	$mfe(\text{scaffold} + rc(\text{ext}) + \text{polyT})$	96%
MFE_3	$mfe(rc(\text{ext}) + \text{polyT})$	100%
MFE_4	$mfe(\text{guide})$	93%
MFE_5	$mfe(\text{guide} + \text{scaffold})$	100%
DeepSpCas9	Valor pré-computado no <code>rawX</code>	100%

As construções de `MFE_1`, `MFE_2` e `MFE_5` não constam do código do projeto nem do texto do artigo, tendo sido descobertas por engenharia reversa das combinações de sequência (*spacer*, *scaffold*, extensão e terminador poli-T). A normalização por *z-score* emprega estatísticas calculadas exclusivamente sobre o conjunto de treino, armazenadas em disco e reaplicadas de forma idêntica em inferência, garantindo a consistência treino-inferência.

Por fim, os índices finais das *features* são {8, 11, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24},

na ordem exata em que o modelo os aprendeu, a qual deve ser rigorosamente preservada em inferência, sob pena de aplicação de pesos a dimensões incorretas, refletida tanto na função de carregamento quanto no módulo de inferência.

2.4.5 Tecnologias utilizadas

A reconstrução das *features* emprega, para as energias de enovelamento, a biblioteca ViennaRNA, por meio de sua função de composição de dobramento, e o módulo `MeltingTemp` do BioPython, com a tabela de vizinhança `R_DNA_NN1`, para as temperaturas de *melting*, refletindo as escolhas adotadas pelo módulo de *score* do projeto original.

2.4.6 Decisões de projeto e desafios

O principal desafio da fase residiu na garantia de consistência absoluta entre as *features* computadas em treino (tabeladas) e em inferência (recalculadas). Um desalinhamento sutil (de unidade, de definição ou de ordem) degradaria a predição sem gerar erro visível. A estratégia adotada para mitigar esse risco foi a validação célula a célula contra o *ground truth* da Tabela S5, que precede qualquer confiança no módulo de inferência. Um segundo desafio, de natureza incremental, foi a decomposição da fase em duas etapas: a primeira, restrita ao treino com *features* tabeladas, mede o ganho puro das *features*; a segunda, dedicada ao recálculo em inferência, viabiliza a operação em sequências novas.

2.4.7 Limitações e impacto

Três *features* de temperatura ($Tm2$, $Tm3$ e ΔTm) não são reproduzíveis em inferência, por dependerem de sequências internas construídas a partir do contexto genômico completo, indisponíveis a partir apenas do PBS e do RTT. Sua exclusão, conforme demonstrado, não representa perda: melhora o desempenho do modelo. Adicionalmente, algumas *features* de energia mínima de enovelamento apresentam fidelidade de reconstrução entre 93% e 99%, constituindo fonte de erro residual de magnitude reduzida.

Quanto ao impacto, a fase eleva o coeficiente de Pearson de 0,42, obtido com a representação exclusivamente sequencial, para 0,74, constituindo o único incremento arquitetural que agrega sinal preditivo substancial e estabelecendo o modelo de produção do projeto.

2.5 Fase 3 — Reconstrução do *encoding* $8 \times L$

2.5.1 Objetivos, motivação e fundamentação

A Fase 3 reconstrói o esquema de *encoding* par-a-par PegRNA-DNA descrito no artigo, uma matriz $8 \times L$ que codifica, por coluna, a identidade nucleotídica, a presença em cada fita, os *mismatches* e a região PAM. Sua inclusão é motivada exclusivamente pela frente de completude com vistas à reprodução de um componente central da arquitetura do

artigo, independentemente de seu efeito sobre a métrica. Esta fase distingue-se, assim, das anteriores, cuja motivação combinava produção e completude.

2.5.2 Fundamentação teórica e relação com o artigo

O esquema do repositório original implementa um *encoding* sete-dimensional distinto do descrito no artigo, operando sobre representações do tipo selvagem ((wild type)) e editado. A especificação do artigo, por sua vez, define uma matriz 8×73 regida por oito regras textuais, cujas quatro primeiras linhas correspondem aos nucleotídeos e cujas quatro últimas codificam, respectivamente, a presença na fita de DNA, a presença na fita de PegRNA, os *mismatches* e a região PAM. A reconstrução seguiu essas oito regras e foi validada célula a célula contra o gabarito da Figura 6B do artigo, com correspondência integral nas oito linhas.

Divergência entre texto e figura do artigo

O texto das regras determina a atribuição do valor 1 para correspondências (*matches*). Entretanto, o gabarito numérico da Figura 6B atribui -1 às correspondências. A convenção real, deduzida da figura, distingue concordância (-1) de divergência ($+1$):

- *Match*: linha do nucleotídeo recebe -1 ;
- *Mismatch*: ambos os nucleotídeos recebem $+1$;
- *Gap*: nucleotídeo presente recebe $+1$.

Verificou-se ainda que a ordem das linhas nucleotídicas segue a sequência A-T-G-C, e não a ordenação alfabética usual. Como na Fase 1, a evidência empírica observada prevaleceu em relação à pura descrição textual.

2.5.3 Modificações por arquivo

A reconstrução do *encoding* envolveu a criação de dois módulos novos, sendo um para as regras de *encoding* e outro para a construção do alinhamento, bem como a modificação de três arquivos existentes para a integração do ramo convolucional correspondente. A Tabela 5 sintetiza as intervenções.

Regras de *encoding* (`encoding_8xL.py`)

Criou-se um módulo dedicado à implementação das oito regras. Sua função principal consiste em receber as sequências de PegRNA e de DNA já alinhadas e retornar a matriz $8 \times L$, aplicando a convenção de sinais deduzida do gabarito. Uma função complementar realiza o preenchimento ou o truncamento da matriz para o comprimento fixo de 73 posições. Incorporou-se, ainda, uma regra (a supressão das linhas de *mismatch* e de presença na fita de PegRNA em posições de PAM que contenham divergência) inferida

Tabela 5: Intervenções por arquivo na Fase 3.

Arquivo	Natureza	Intervenção
<code>encoding_8xL.py</code>	Novo	Implementação das oito regras de <i>encoding</i> $8 \times L$, com a convenção de sinais deduzida do gabarito.
<code>alignment_73.py</code>	Novo	Construção do alinhamento PegRNA-DNA de 73 posições e geração da matriz de <i>encoding</i> por candidato.
<code>train_models.py</code>	Modificado	Ramo convolucional na classe do Transformer e passagem da matriz no laço de treino.
<code>read_data.py</code>	Modificado	Geração da coluna de <i>encoding</i> na função de carregamento.
<code>evaluate.py</code>	Modificado	Passagem da matriz de <i>encoding</i> à chamada do modelo na rotina de avaliação.

de um único exemplo do gabarito e, por isso, documentada como aproximação sujeita a refinamento.

Construção do alinhamento de 73 posições (`alignment_73.py`)

O artigo afirma que as sequências PegRNA e DNA possuem comprimento igual a 73, porém não especifica a composição dessas posições nem o procedimento de alinhamento. Criou-se, portanto, um módulo responsável por essa construção, que expõe uma função de montagem do par alinhado a partir do *wide target* e da extensão 3', e uma função que encadeia essa montagem com o *encoding*, retornando a matriz 8×73 pronta para o modelo.

Alinhamento *on-target*

Na ausência de especificação do artigo, adotou-se a decomposição [*wide target* 47] + [extensão 3' até 26] = 73. No cenário *on-target*, o par PegRNA-DNA é complementar por construção; a implementação adota, por conseguinte, sequências idênticas nas posições reais, o que anula *mismatches* espúrios e reflete corretamente a ausência de desalinhamento. Como consequência, as linhas de *mismatch* e de *gap* permanecem triviais, e a matriz codifica essencialmente a composição de sequência e a região PAM. Ambas as decisões, a decomposição de 73 posições e a identidade das sequências no *on-target*, constituem decisões de projeto aqui documentadas, e não reproduções literais do método original.

Ramo convolucional na arquitetura (`train_models.py`)

A classe `TransformerEncoderDecoderModelOrder3` foi estendida com um ramo convolucional destinado ao processamento da matriz de *encoding*. No método `__init__`, introduziu-se um parâmetro de dimensão de *encoding* que, quando positivo, instancia duas camadas convolucionais unidimensionais, uma operação de *pooling* adaptativo e uma

projeção linear; a dimensão de entrada da camada de decisão passou a acomodar também a saída desse ramo. No método `forward`, a assinatura passou a receber a matriz de *encoding*, processada pelo ramo convolucional e concatenada aos *k-mers* e às *features* escalares antes da camada de decisão. A rotina de treino da mesma unidade foi ajustada para construir o tensor da matriz a partir do *batch* e repassá-lo à chamada do modelo.

Defeito corrigido: dupla invocação da camada de decisão

Durante a integração do ramo convolucional, constatou-se que a camada totalmente conectada era invocada antes da concatenação da saída do ramo, o que produzia incompatibilidade dimensional. A correção assegurou uma única invocação da camada de decisão, posterior à agregação completa de todos os vetores — *k-mers*, *features* escalares e *encoding*. À semelhança das fases anteriores, as referências à dimensão do *encoding* foram protegidas por acesso condicional, garantindo a retrocompatibilidade com modelos serializados antes desta intervenção.

Geração de *encoding* no carregamento (`read_data.py`)

A função de carregamento foi estendida para gerar, por registro, uma coluna adicional contendo a matriz de *encoding*, produzida a partir do *wide target* e das sequências de PBS e RTT do candidato. Essa geração ocorre em conjunto com a produção das colunas de *tokens* e de *features*, de modo que cada registro carregue simultaneamente todas as representações exigidas pelos ramos da arquitetura.

Passagem de *encoding* na avaliação (`evaluate.py`)

De forma simétrica ao laço de treino, a rotina de avaliação foi ajustada para construir o tensor da matriz de *encoding* a partir do *batch* de teste e repassá-lo à chamada do modelo, preservando a simetria entre os caminhos de treino e de avaliação.

2.5.4 Fluxo de execução, tecnologias e decisões de projeto

O fluxo consolidado compreende, em treino, a geração da matriz 8×73 de *encoding* no carregamento (`read_data.py`), seu processamento pelo ramo convolucional (`train_models.py` - duas camadas `Conv1d` seguidas de *pooling* adaptativo e projeção linear, constituindo aproximadamente ~ 16 mil parâmetros) cuja saída é concatenada aos *k-mers* e às *features* escalares antes do MLP (*Multilayer Perceptron*). Em inferência, a construção do par alinhado e da matriz correspondente (`alignment_73.py`, `encoding_8xL.py`). A validação do *encoding* foi conduzida contra o gabarito da Figura 6B do artigo, de modo análogo ao papel desempenhado pela Tabela S5 na validação dos atributos da Fase 2.

A arquitetura precisa do ramo convolucional (número de filtros, dimensão do *kernel* e configuração do *pooling*) não é especificada pelo artigo, constituindo decisão de projeto.

Adotou-se uma configuração deliberadamente simples, de reduzido número de parâmetros, a fim de minimizar a introdução de capacidade excedente.

2.5.5 Limitações, impacto e análise crítica

O resultado empírico demonstra degradação do desempenho, com redução do coeficiente de Pearson de 0,74 para 0,67. A interpretação é consistente com a natureza do esquema: concebido para a detecção de *off-targets*, cenário em que *mismatches* e *bulges* são informativos, o esquema não agrega informação no cenário *on-target*, onde tais linhas são triviais e as linhas nucleotídicas redundam com os *k-mers*. O ramo convolucional adiciona capacidade de *overfitting* sem sinal correspondente, evidenciada pela antecipação da melhor época de 83 para 59.

É necessário ressaltar que ainda permanecem como limitações: a decomposição de 73 posições e a convenção de identidade das sequências no *on-target*, adotadas na ausência de especificação do artigo; a regra de canto do PAM, inferida de exemplo único; e a arquitetura do ramo convolucional, definida por decisão de projeto. Nenhuma dessas limitações afeta o modelo de produção, que dispensa a *encoding* $8 \times L$. A fase cumpre, todavia, seu propósito de completude ao reproduzir fielmente o esquema de *encoding* e quantifica, de forma empírica, sua contribuição nula, ou levemente adversa, à predição de eficiência *on-target*.

2.6 Fase 4 — Camada de *embedding* DNABERT

2.6.1 Objetivos, motivação e fundamentação

A Fase 4 reproduz a camada de *embedding* baseada em DNABERT, um modelo BERT [4] pré-treinado no genoma humano com tokenização em *k-mers* de comprimento 6. Sua inclusão insere-se na frente de completude. O artigo reporta que o DNABERT, o DistilBERT-DNABERT [10] e um Transformer de três camadas atingem desempenho praticamente idêntico (Pearson $\sim 0,74$), sugerindo, já na fundamentação, contribuição marginal. Esta fase foi conduzida em duas etapas sucessivas — extração congelada (*frozen extraction*) e *fine-tuning* —, refletindo o equilíbrio entre a viabilidade prática e a fidelidade ao método original.

2.6.2 Viabilização e desafios de infraestrutura

A execução do DNABERT exigiu a superação de obstáculos de infraestrutura anteriores a qualquer intervenção no código do projeto. A biblioteca `transformers`, na versão 2.5 empacotada pelo repositório do DNABERT, expõe uma interface de programação distinta das versões correntes: a tokenização realiza-se por um método específico de *encoding* em lote, não sendo o *tokenizer* diretamente invocável, e a saída do modelo apresenta-se como

tupla, cujo primeiro elemento contém os estados ocultos. Some-se que a obtenção dos pesos pré-treinados requer a recuperação explícita dos arquivos versionados por *Git LFS*, sem a qual o repositório fornece apenas os ponteiros correspondentes. A verificação de viabilidade compreendeu a confirmação do carregamento do modelo e a medição de seu consumo de memória, que se revelou reduzido em modo de inferência, o que viabilizou o pré-cálculo dos *embeddings* descrito adiante.

2.6.3 Modificações por arquivo

A reprodução do DNABERT envolveu a criação de dois módulos novos, sendo um para a extração de *embeddings*, outro para seu pré-cálculo, bem como a modificação de três arquivos existentes para a integração do ramo correspondente à arquitetura. As intervenções diferem conforme a modalidade: a abordagem *frozen* baseia-se em *embeddings* pré-computados, ao passo que o *fine-tuning* incorpora o modelo BERT como submódulo treinável. A Tabela 6 sintetiza as intervenções.

Tabela 6: Intervenções por arquivo na Fase 4.

Arquivo	Natureza	Intervenção
<code>dnabert_embed.py</code>	Novo	Tokenização em 6-mers e extração de <i>embeddings</i> do DNABERT, com <i>pooling</i> pelo <i>token</i> de classificação.
<code>precompute_embeddings.py</code>	Novo	Pré-cálculo dos <i>embeddings</i> de todos os registros, para a abordagem congelada.
<code>train_models.py</code>	Modificado	Ramo DNABERT na classe do Transformer (métodos <code>__init__</code> e <code>forward</code>), com bifurcação entre as modalidades, e grupos de taxa de aprendizado diferenciados.
<code>read_data.py</code>	Modificado	Geração da coluna de <i>embeddings</i> (<i>frozen</i>) ou de <i>tokens</i> (<i>fine-tune</i>) na função de carregamento.
<code>evaluate.py</code>	Modificado	Passagem dos <i>embeddings</i> ou <i>tokens</i> à chamada do modelo na rotina de avaliação.

Extração de *embeddings* (`dnabert_embed.py`)

Criou-se um módulo dedicado à interface com o DNABERT. Expõe uma função de conversão de uma sequência de nucleotídeos em 6-mers sobrepostos e uma classe responsável por carregar o modelo e o *tokenizer*, mantendo o modelo em modo de avaliação e com gradientes desabilitados. O método de extração dessa classe processa um lote de sequências e retorna, por sequência, o vetor de estado oculto do *token* de classificação. A implementação observa as particularidades da interface da versão 2.5 da biblioteca, tanto no *batch encoding* quanto no acesso à saída do modelo como tupla.

Pré-cálculo dos *embeddings* (`precompute_embeddings.py`)

Como, na abordagem *frozen*, os *embeddings* não variam entre épocas, criou-se um módulo que os pré-computa uma única vez para a totalidade dos registros, gravando o resultado em disco na mesma ordem do conjunto de dados. Essa decisão, alinhada à frente de produção, elimina a passagem pelo modelo BERT durante o treino, reduzindo substancialmente seu custo computacional.

Ramo DNABERT na arquitetura (`train_models.py`)

A classe `TransformerEncoderDecoderModelOrder3` foi estendida para acomodar ambas as modalidades de integração do DNABERT — a extração congelada e o *fine-tuning* —, preservando ainda o caminho de execução sem o ramo. No método `__init__`, um parâmetro de dimensão do *embedding*, quando positivo, instancia uma projeção linear e sua ativação; um parâmetro adicional de *fine-tuning* determina se o modelo BERT é incorporado como submódulo treinável, carregado a partir dos pesos pré-treinados. Uma vez que a saída do ramo DNABERT é concatenada às demais representações antes da camada final, a dimensão de entrada dessa camada foi ampliada na medida exata da projeção acrescentada, de modo que a rede permanece dimensionalmente consistente tanto com o ramo ativo quanto sem ele. No método `forward`, a assinatura foi ampliada para receber, conforme a modalidade, quer o *embedding* pré-computado, quer os identificadores e a máscara de atenção dos *tokens*; um desvio condicional seleciona o caminho apropriado, processando os *tokens* pelo BERT quando em *fine-tuning*, ou projetando diretamente o *embedding* quando congelado. A saída é concatenada às demais representações antes da camada de decisão.

Taxas de aprendizado diferenciadas

No *fine-tuning*, a rotina de treino foi ajustada para constituir grupos de parâmetros distintos no otimizador, isto é, uma taxa de aprendizado reduzida para os parâmetros do submódulo BERT e a taxa usual para o restante do modelo. Essa diferenciação preserva os pesos pré-treinados de uma desestabilização por atualização excessiva, tal como prática estabelecida em literatura no contexto do ajuste fino de modelos de linguagem [8].

Geração de *embeddings* ou *tokens* no carregamento (`read_data.py`)

A função de carregamento foi estendida de forma condicional à modalidade. Na abordagem *frozen*, associa a cada registro o *embedding* pré-computado correspondente, recuperado por indexação posicional sobre o arquivo gravado — para o que se preserva a posição original de cada registro antes de qualquer partição, evitando desalinhamento entre o registro e seu *embedding*. Na abordagem de *fine-tuning*, instancia-se uma única vez o *tokenizer* e gera, por registro, os identificadores e a máscara de atenção dos 6-mers da sequência-alvo.

Passagem de *embeddings* ou *tokens* na avaliação (`evaluate.py`)

De forma simétrica ao laço de treino, a rotina de avaliação foi ajustada para construir, a partir do *batch* de teste, quer o tensor de *embeddings*, quer os tensores de identificadores e de máscara, repassando-os à chamada do modelo conforme a modalidade em uso.

2.6.4 Duas abordagens e decisões de projeto

Duas modalidades de integração foram implementadas e comparadas, refletindo o equilíbrio entre produção e completude:

- **Abordagem 1 (*frozen*):** O DNABERT opera como extrator de *embeddings* fixos de modo que, como os vetores não variam entre épocas, tendo sido pré-computados uma única vez, o que reduz drasticamente o custo de treino e prioriza a produção.
- **Abordagem 2 (*fine-tuning*):** O DNABERT torna-se submódulo treinável, ajustado conjuntamente à tarefa, como descrito no artigo, exigindo-se taxas de aprendizado diferenciadas (2×10^{-5} para o BERT, 1×10^{-4} para o restante) a fim de preservar os pesos pré-treinados, priorizando-se a completude e a fidelidade ao artigo.

2.6.5 Sequência processada e fluxo

Em ambas as abordagens, submeteu-se ao DNABERT a sequência *wide target* de 47 pb, tokenizada em 6-*mers* sobrepostos, com *pooling* pelo *token* de classificação. Ambas as abordagens resultam em desempenho inferior ao da Fase 2, com o *fine-tuning* recuperando fração marginal sobre a versão congelada.

Enquanto o fluxo da abordagem *frozen* compreende o pré-cálculo dos *embeddings* (`precompute_embeddings.py`), sua indexação no carregamento e sua projeção no ramo da arquitetura; o fluxo do *fine-tuning* compreende a tokenização no carregamento, a passagem dos *tokens* pelo submódulo BERT durante o treino e a atualização conjunta de seus pesos.

2.6.6 Limitações e impacto

Ambas as abordagens resultam em desempenho inferior ao da Fase 2. A extração *frozen* conduz a um coeficiente de Pearson de 0,71, ao passo que o *fine-tuning* recupera fração marginal, alcançando 0,72 — em ambos os casos, abaixo do valor de 0,74 da Fase 2. O resultado confirma empiricamente a indicação efetuada no próprio artigo de que a camada DNABERT, ainda quando ajustada, não supera o modelo que a dispensa, pois a informação de sequência que codifica já é capturada pelos *k-mers* de ordens 1, 2 e 3. A antecipação da melhor época no *fine-tuning* (de 83 para 67) corrobora a hipótese de *overfitting* sem sinal correspondente.

Registra-se, como decisão de escopo, que se submeteu ao DNABERT apenas a sequência *wide target*, e não também a extensão 3'; tal restrição pode explicar a diferença residual em relação ao valor reportado no artigo, sem, contudo, alterar a conclusão. A fase cumpre integralmente seu propósito de completude ao reproduzir o componente DNABERT em suas duas modalidades (inclusive a de *fine-tuning*, fiel ao artigo) e quantifica empiricamente sua contribuição nula à predição de eficiência.

2.7 Fase 5 — Protocolos de validação

2.7.1 Objetivos e motivação

A Fase 5 estabelece a avaliação rigorosa do modelo, mediante três protocolos que medem aspectos distintos da capacidade preditiva. Sua motivação combina as duas frentes: à produção, por fornecer uma métrica confiável e estável do modelo de operação; à completude, por introduzir uma transparência metodológica superior à do artigo, capaz de distinguir a interpolação intra-alvo da generalização real a novos *loci* genômicos. Distintamente das fases anteriores, a Fase 5 caracteriza-se menos pela modificação de componentes existentes e mais pela criação de rotinas de avaliação dedicadas.

2.7.2 Fundamentação teórica e relação com o artigo

O artigo avalia o modelo por validação cruzada *k-fold* ($k = 5$), particionada por registro, contudo, não distingue registros que compartilham o mesmo alvo genômico, de modo que a investigação da estrutura do conjunto de dados revelou que os 43.149 registros correspondem a apenas 1.956 alvos únicos — em média, ~ 22 extensões por alvo. Sob particionamento por registro, praticamente a totalidade dos alvos do conjunto de teste comparece também no conjunto de treino, configurando vazamento estrutural, de maneira que o modelo interpola entre extensões de alvos já observados, em vez de generalizar para *loci* inéditos.

Validação diferenciada por *loci*

Para medir a generalização real, adotou-se o protocolo GroupKFold, particionado por alvo, que garante a ausência total de alvos compartilhados entre treino e teste. Esse protocolo caracteriza o cenário de uso efetivo da ferramenta, isto é, a predição sobre *loci* não observados durante o treino, e não é reportado no artigo original, constituindo uma contribuição metodológica do presente trabalho.

2.7.3 Modificações por arquivo

A caracterização dos três protocolos envolveu a criação de rotinas de avaliação dedicadas e a reutilização de componentes consolidados nas fases anteriores. A Tabela 7 sintetiza

as intervenções.

Tabela 7: Intervenções por arquivo na Fase 5.

Arquivo	Natureza	Intervenção
<code>avaliar_fase5.py</code>	Novo	Avaliação por retenção sobre o conjunto HT-Test, com cômputo dos coeficientes de correlação.
<code>fase5_cv.py</code>	Novo	Validação cruzada <i>k-fold</i> particionada por registro, sobre o conjunto de dados completo.
<code>group_kfold_cv.py</code>	Novo	Validação cruzada particionada por alvo, com identificador de grupo e salvamento incremental por <i>fold</i> .
<code>evaluate.py</code>	Reutilizado	Rotina de avaliação do modelo, já adaptada nas fases anteriores para as representações adicionais.
<code>read_data.py</code>	Reutilizado	Função de carregamento, empregada para a montagem do conjunto completo e das partições.

Avaliação por *holdout* (`avaliar_fase5.py`)

Criou-se uma rotina de avaliação sobre o conjunto de *holdout* (HT-Test), que aplica o modelo treinado às amostras reservadas e computa os coeficientes de correlação de Pearson e de Spearman, além de medidas de erro absoluto e quadrático. Por operar em modo de inferência, sobre um conjunto de dimensão reduzida, essa rotina constitui uma via rápida de iteração, empregada ao término de cada fase para a medição de sua contribuição isolada.

Validação cruzada por registro (`fase5_cv.py`)

Criou-se uma rotina de validação cruzada *k-fold*, particionada por registro sobre o conjunto de dados completo, seguindo-se a metodologia do artigo. Cada *fold* treina um modelo do zero e o avalia sobre a partição reservada, reportando-se a média e o desvio padrão dos coeficientes ao final. Essa rotina incorre em maior custo computacional, reservando-se seu uso à obtenção da métrica robusta do modelo consolidado.

Validação cruzada por alvo (`group_kfold_cv.py`)

Criou-se uma rotina de validação cruzada particionada por alvo, que emprega a representação tokenizada da sequência-alvo como identificador de grupo — sequências idênticas geram identificadores idênticos, assegurando que todas as extensões de um mesmo alvo recaiam exclusivamente sobre uma única partição. A rotina verifica, explicitamente, a ausência de alvos compartilhados entre treino e teste em cada *fold*, e realiza o salvamento incremental dos resultados parciais, de modo a preservar o progresso ante eventual interrupção de execuções prolongadas. Aceita, ainda, a especificação de um subconjunto de *folds*, o que permite confirmar a tendência de generalização a custo reduzido, sem a

execução da totalidade dos *folds*.

Normalização e artefatos por partição

Cada protocolo emprega arquivos de estatísticas de normalização e de modelos serializados com nomenclatura própria, de modo a evitar colisões entre execuções concomitantes ou sucessivas. A validação sem vazamento, em particular, deriva as estatísticas de normalização da partição de treino de cada *fold*, preservando o princípio de que nenhuma informação do conjunto de teste influencie a preparação das *features*.

2.7.4 Fluxo de execução

Os três protocolos compartilham a função de carregamento (`read_data.py`) e a rotina de avaliação (`evaluate.py`), diferindo no procedimento de particionamento. A validação por *holdout* opera sobre a divisão nativa do conjunto de dados; a validação por registro emprega o particionamento aleatório da totalidade dos registros; e a validação por alvo emprega o particionamento por grupo, mediante a funcionalidade correspondente do Scikit-learn [9]. A verificação da ausência de vazamento foi conduzida de forma independente do treino, confirmando-se a inexistência de alvos compartilhados entre as partições sob o protocolo por grupo, em contraste com a sobreposição praticamente integral sob o protocolo por registro.

2.7.5 Limitações e impacto

A aplicação dos três protocolos ao modelo de produção revela uma progressão decrescente dos coeficientes, da interpolação intra-alvo à generalização real, cuja discussão quantitativa é apresentada na Seção 3. A principal implicação metodológica é a constatação de que a métrica reportada pelo artigo, obtida por particionamento por registro, está sujeita ao vazamento estrutural aqui caracterizado, de modo que a generalização real a *loci* inéditos é plausivelmente inferior ao valor reportado.

Como limitação, registra-se que a validação sem vazamento foi conduzida sobre um subconjunto dos *folds*, suficiente para confirmar a estabilidade da tendência (atestada pela reduzida dispersão entre *folds*), porém não sobre a totalidade deles, decisão motivada pelo custo computacional. O impacto desta fase é duplo ao (i) fornecer a métrica robusta que consolida o modelo de produção e (ii) estabelecer um protocolo de avaliação mais rigoroso que o do artigo, apto a servir de referência para trabalhos futuros.

2.8 Fase 0 — Pré-treino em ClinVar

2.8.1 Objetivos e motivação

A Fase 0 corresponde à etapa de pré-treino descrita no artigo, anterior ao ajuste supervisionado sobre os dados de eficiência. Sua motivação, no método original, é dotar o modelo de uma representação inicial das sequências de pegRNA aprendida sobre um vasto conjunto de variantes genômicas, de modo que o ajuste fino subsequente parta de pesos já informados pela estrutura do domínio. Esta fase recebe a numeração zero por uma razão conceitual relevante: embora tenha sido a última a ser considerada na ordem de desenvolvimento, ocuparia a primeira posição na ordem de execução do *pipeline* final, precedendo todas as demais. Essa distinção entre a ordem de escrita e a ordem de execução, discutida na Seção 2.2, atinge aqui sua expressão máxima.

2.8.2 Fundamentação teórica e relação com o artigo

O artigo descreve a geração de aproximadamente 442 milhões de pegRNAs candidatos, subdivididos entre construções corretivas e de instalação de edição, a partir de um conjunto da ordem de dezenas de milhares de variantes patogênicas do ClinVar [6], referenciadas ao genoma humano. Sobre esse volume de sequências, o modelo é pré-treinado antes do ajuste fino nos dados de eficiência. O propósito preciso do pré-treino, contudo, não é especificado com o detalhamento necessário à sua reprodução não explicitando-se o objetivo de otimização adotado (por exemplo, a modelagem de linguagem mascarada), tampouco os hiperparâmetros ou os critérios de convergência.

2.8.3 Análise de viabilidade

A avaliação da reprodutibilidade desta fase conduziu à conclusão de irreprodutibilidade prática, sustentada por três obstáculos independentes e convergentes.

- **Escala dos dados:** A geração e o processamento de centenas de milhões de sequências candidatas impõem requisitos de computação e de armazenamento incompatíveis com o tempo disponível à execução o projeto.
- **Indisponibilidade dos dados:** O conjunto de pegRNAs candidatos não é distribuído com o material do artigo, e sua reprodução fiel dependeria da replicação exata do procedimento de derivação a partir das variantes do ClinVar, não integralmente especificado.
- **Subespecificação do objetivo:** A ausência de definição precisa do objetivo de pré-treino impede a reprodução do procedimento, ainda que os dados estivessem disponíveis, pois o alvo de otimização permaneceria indeterminado.

Limite de reprodutibilidade

Ao contrário das limitações pontuais registradas nas Fases 1 a 4, supríveis por decisões de projeto documentadas, a Fase 0 constitui um limite de reprodutibilidade de natureza estrutural, decorrente simultaneamente da escala dos dados, de sua indisponibilidade e da subespecificação do método. Documenta-se, portanto, esta fase como não implementada, distinguindo-se explicitamente o que não foi reproduzido por inviabilidade daquilo que foi adaptado por decisão de projeto.

2.8.4 Proposta de versão factível

Embora o pré-treino original seja irreproduzível em sua escala, delineia-se uma versão factível, concebida para preservar o princípio metodológico do pré-treino, isto é, a aquisição de uma representação inicial informada pelo domínio, sem, contudo incorrer em seus custos proibitivos. A proposta compreende quatro etapas sequenciais:

1. **Subamostragem de variantes:** Seleção de um subconjunto de variantes patogênicas do ClinVar em ordem de grandeza computacionalmente viável para esse projeto (na ordem de milhares, e não das dezenas de milhares), preservando a diversidade de contextos genômicos.
2. **Geração dos candidatos correspondentes:** Derivação dos pegRNAs candidatos para o subconjunto selecionado, reutilizando a lógica de reconstrução do *wide target* consolidada na Fase 1.
3. **Pré-treino auto-supervisionado:** Aplicação de um objetivo de modelagem de linguagem mascarada sobre as sequências geradas, produzindo pesos iniciais informados pela estrutura das sequências.
4. ***Fine-tuning e transfer-learning:*** Inicialização do modelo de eficiência com os pesos pré-treinados e ajuste supervisionado sobre os dados de eficiência, mensurando-se o eventual ganho atribuível à transferência.

Decisão de sequenciamento

A versão factível, caso implementada, deve ser desenvolvida por último, com a arquitetura já consolidada pelas fases anteriores, ainda que ocupe a primeira posição na execução do *pipeline*. Essa ordenação decorre do princípio de decomposição incremental em que o efeito do pré-treino só é interpretável sobre uma arquitetura estável, cujo desempenho sem pré-treino já esteja estabelecido como *baseline*. A distinção entre o momento de escrita e o momento de execução é, portanto, uma decisão metodológica deliberada, e não um acidente de cronograma.

2.8.5 Impacto e considerações finais

A não implementação da Fase 0 não compromete o modelo de produção nem as conclusões do trabalho. As evidências acumuladas nas fases anteriores, em particular a constatação de que apenas os atributos escalares agregam sinal preditivo, sugerem que o ganho marginal de um pré-treino auto-supervisionado, sobre uma arquitetura cujo desempenho já se aproxima ou supera o do artigo, seria provavelmente reduzido. Ademais, o artigo reporta que a arquitetura sem os componentes de *embedding* contextual atinge desempenho equivalente ao das variantes que os incorporam, o que enfraquece a expectativa de contribuição substancial do pré-treino. A Fase 0 permanece, assim, documentada como limite de reprodutibilidade e como direção de trabalho futuro, cuja versão factível é delineada mas cuja implementação transcende o escopo e os recursos do presente projeto.

2.9 Interface Gráfica

O artigo de referência disponibiliza o DTMP-Prime como conjunto de *scripts* de linha de comando, cuja operação pressupõe familiaridade com ambientes Python, edição manual de arquivos de configuração e manipulação de formatos de arquivos comuns em bioinformática. Uma vez que o presente trabalho tem, entre seus objetivos, melhorar a acessibilidade dessa ferramenta a pesquisadores sem formação computacional, desenvolveu-se uma interface gráfica que se encontrava subdesenvolvida e inoperante no repositório original.

2.9.1 Arquitetura

A interface organiza-se em duas camadas. A camada de serviço, implementada em Flask, expõe um conjunto reduzido de rotas: submissão de tarefa, consulta de estado, recuperação de resultados e transferência de arquivos gerados. A execução do *pipeline* ocorre de forma assíncrona, em processo apartado, de modo que a sessão do usuário não permanece bloqueada durante a busca de candidatos, que pode estender-se por minutos. A camada de apresentação consiste em um documento HTML único, sem dependências de compilação, o que preserva a característica de instalação simples do projeto.

2.9.2 Contribuições em relação ao trabalho original

As adições podem ser agrupadas em quatro frentes.

Entrada de dados sem pré-processamento

O *pipeline* original admite exclusivamente arquivos VCF (*Variant Call Format*) acompanhados de um genoma de referência indexado, o que exige que o usuário disponha de ambos e saiba produzi-los. A interface atual foi então desenvolvida de modo a admitir a especificação direta e manual para uma única variante de pares de sequências selvagem e mutante, bem como a serialização de múltiplas variantes a partir de um arquivo FASTA,

dispensando a manipulação de arquivos VCF. Um painel de comparação exibe a diferença detectada entre as sequências fornecidas, permitindo que o usuário confirme visualmente que a edição pretendida corresponde à que será projetada, antes de submeter à execução.

Configuração dos parâmetros

Os parâmetros do arquivo de configuração foram organizados em grupos temáticos — PBS, RTT, guia de nickagem e *protospacer* —, com valores padrão pré-definidos e rótulos redigidos em termos de domínio, e não de nomes de variáveis. O usuário pode, assim, executar a ferramenta sem editar qualquer arquivo, mantendo ainda a possibilidade de ajuste fino quando necessário.

Apresentação progressiva dos resultados

A saída original consiste em uma tabela de largura considerável, cuja interpretação exige conhecimento prévio do significado de cada coluna. A interface substitui essa apresentação por um percurso em etapas: seleção da variante, seleção do pegRNA, seleção do guia de nickagem e resumo consolidado do desenho escolhido, com as sequências do *protospacer*, PBS, RTT e extensão 3' exibidas com destaque por composição de bases. A tabela completa permanece acessível, e os arquivos de resultado podem ser transferidos em seu formato original, de modo que nenhuma informação produzida pelo *pipeline* é suprimida.

Rastreabilidade da execução

O registro de execução do *pipeline* é transmitido à interface e apresentado com marcação por severidade, permitindo que o usuário identifique avisos que, na operação por linha de comando, se perderiam no fluxo de saída padrão. Mantém-se ainda um histórico local de tarefas submetidas, com seus identificadores, estados e descrições, o que possibilita recuperar resultados de execuções anteriores sem repetir o processamento.

2.9.3 Tutorial

A interface web do DTMP-Prime executa todo o pipeline de desenho e ranqueamento de pegRNAs sem exigir linha de comando. A tela divide-se em três regiões fixas: o *cabeçalho* (identificação e indicador de status do *job*), a *barra lateral* (navegação, agrupada em **Análise** e **Suporte**, com o botão **Executar** ao pé) e a *área principal*, que exibe a aba selecionada. Os passos a seguir cobrem um fluxo completo, da entrada dos dados ao download dos resultados.

1. **Navegação:** Na barra lateral, o grupo **Análise** reúne as abas **Entrada**, **Configuração** e **Resultados**; o grupo **Suporte** reúne **Tutorial**, **Informações** e **Histórico**. O indicador no cabeçalho reflete o estado atual (**Pronto**, **Executando**, **Concluído** ou **Erro**).
2. **Fornecimento das variantes:** Navegue até à aba **Entrada** e escolha um dos dois modos de entrada no seletor no topo da aba:

- **Sequências manuais:** Para cada variante, preencha um nome identificador e as duas sequências: **Seq. Referência (ref)**, com o alelo original, e **Seq. Editada (alt)**, com a edição já incorporada. A interface detecta automaticamente o tipo de edição (substituição, inserção ou deleção) e sua posição. Use + **Adicionar variante** para processar várias variantes em lote e o botão de remoção para descartá-las.
- **Arquivo FASTA:** Arraste um arquivo **.fa/.fasta** para a área indicada (ou clique para selecioná-lo). Cada variante deve ser representada por um par de registros, com os sufixos **_ref** e **_alt** no cabeçalho e na linha seguinte as sequências devem estar na orientação 5' → 3' e conter apenas os nucleotídeos A, T, G e C (sem códigos de ambiguidade IUPAC). Além disso, é recomendado incluir contexto suficiente em torno do sítio editado (≈ 200) pb para que o pipeline encontre protospacers e PAMs adequados. A seguir, tem-se um exemplo do formato FASTA aceito com os pares **ref/alt** por variante:

```

1 >CFTR_F508del_ref
2 AAAATATCATCTTTGGTGTTCCTATGATGAATATAGATACAGAAGCGT
3 >CFTR_F508del_alt
4 AAAATATCATTTGGTGTTCCTATGATGAATATAGATACAGAAGCGTCAT

```

3. **Ajuste dos parâmetros (aba Configuração):** Opcionalmente, navegue até à aba **Configuração** para ajustar os parâmetros de inferência. Os valores padrão são definidos pela configuração existente no repositório original, organizados em quatro grupos:
 - **PBS:** comprimentos mínimo e máximo, em nt.
 - **RTT:** comprimentos mínimo e máximo e distância mínima a partir da extremidade 5' do RTT.
 - **sgRNA / ngRNA:** espaço de busca (pb), comprimento do sgRNA, sequência PAM (padrão NGG), *offset* do corte da Cas9 (padrão -3), distância máxima do ngRNA (para PE3) e distância máxima entre a mutação e o sgRNA.
 - **Execução:** número de *jobs* paralelos (**n_jobs**) e nível de **debug** (sempre ≥ 1).
4. **Execução da inferência:** Com a entrada definida, clique em **Executar** na porção inferior da barra lateral. A interface valida os campos, envia o *job* ao servidor e redireciona automaticamente para a aba **Resultados**.
5. **Progresso da execução:** Enquanto o status for **Executando**, o **Log de execução** exibe, em tempo real, o comando disparado e as mensagens do pipeline. Ao término, o cabeçalho passa a **Concluído** e os resultados são apresentados; em caso de falha, o status passa a **Erro** e o log indica a causa.

6. **Resultados:** Navegue até à aba **Resultados** Um painel de estatísticas resume a análise (número de variantes e quantas produziram desenhos PE2, PE3, PE3b e dPAM). Logo abaixo, um fluxo em três etapas conduz a inspeção:
 - (a) **Candidatos a pegRNA:** Tabela com todos os pegRNAs ordenados por eficiência predita. É possível filtrar por texto e reordenar por qualquer coluna (*spacer*, modo, PBS, RTT, comprimentos, fita, entre outras). Selecione a linha do pegRNA desejado para avançar.
 - (b) **Nicking guide:** Para desenhos PE3/PE3b, escolha ngRNA associado ao pegRNA selecionado; em modo PE2, esta etapa é apenas informativa.
 - (c) **Resumo do desenho:** Consolida o pegRNA e o ngRNA escolhidos, com *spacer*, PBS, RTT, extensão 3' e demais parâmetros do desenho final.
7. **Download:** No topo da aba **Resultados**, os botões **Top pegRNAs** e **Sumário** exportam, respectivamente, a tabela completa de candidatos e o resumo por variante. Os demais artefatos gerados na execução também ficam disponíveis para download; a aba **Informações** descreve o conteúdo de cada arquivo.
8. **Histórico:** Navegue até à aba **Histórico**, onde cada execução é registrada, com data, descrição e status. Use **Ver** para reabrir os resultados de um *job* concluído sem reexecutá-lo. O registro persiste localmente, de modo que análises passadas permanecem acessíveis mesmo após reiniciar o servidor.

Solução de problemas

Se a execução terminar em **Erro**, consulte o **Log de execução**: as causas mais comuns são dependências ausentes, configuração inválida ou sequências **ref/alt** malformadas. Aumentar o nível de **debug** registra mais mensagens de diagnóstico e pode colaborar para a identificação da origem do problema.

3 Resultados e Discussão

3.1 Quadro consolidado das medições por fase

A Tabela 8 apresenta o desempenho isolado de cada componente arquitetural, medido sobre o conjunto de teste (HT-Test) segundo o protocolo de *holdout*. A estratégia de decomposição incremental permite atribuir de forma inequívoca o efeito de cada camada.

3.2 Resultado central

O resultado mais significativo refere-se à forte evidência de que, entre as camadas arquiteturais propostas, apenas as *features* escalares agregam sinal preditivo relevante pois a

Tabela 8: Contribuição isolada de cada fase (conjunto HT-Test, *holdout*).

Fase	Componente	Pearson	Spearman	Efeito
1	<i>wide target</i> (sequência)	0,42	0,47	<i>Baseline</i>
2	+ 14 <i>features</i> escalares	0,74	0,78	Ganho significativo
3	+ codificação $8 \times L$	0,67	0,74	Degradação
4a	+ DNABERT (<i>frozen</i>)	0,71	0,77	Degradação leve
4b	+ DNABERT (<i>fine-tuned</i>)	0,72	0,77	Degradação leve

Fase 2, isoladamente, praticamente dobra o coeficiente de Pearson (de 0,42 para 0,74), enquanto as Fases 3 e 4, ambas reproduzidas com fidelidade verificada contra gabaritos do artigo, não somente deixam de agregar como degradam levemente o desempenho.

Esta constatação encontra fundamento na própria estrutura das representações. As *features* escalares (*DeepSpCas9*, energias de enovelamento, conteúdo GC) codificam propriedades termodinâmicas e enzimáticas não recuperáveis da sequência isolada, ao passo que o *encoding* $8 \times L$ e os *embeddings* DNABERT codificam informação de sequência já capturada pelos *k-mers* de ordens 1, 2 e 3. A adição dessas componentes introduz, portanto, capacidade de *overfitting* sem sinal correspondente, hipótese corroborada pela sistemática antecipação da melhor época nas fases que degradam o desempenho (Tabela 9).

Tabela 9: Melhor época registrada por treino e sua relação com o desempenho. Fases que degradam o desempenho em relação à Fase 2 apresentam melhor época antecipada, indicando *overfitting* precoce.

Fase	Componente	Melhor época	Pearson	Relação com a Fase 2
2	14 <i>features</i> (produção)	83	0,74	<i>Baseline</i>
3	+ codificação $8 \times L$	59	0,67	Antecipa (−24)
4a	+ DNABERT (<i>frozen</i>)	88	0,71	Próxima (+5)
4b	+ DNABERT (<i>fine-tuned</i>)	67	0,72	Antecipa (−16)

Interpretação do papel da codificação e do DNABERT

O *encoding* $8 \times L$ foi concebido, no artigo, para a predição de *off-targets*, cenário em que *mismatches* e *bulges* são informativos. Sua aplicação à predição de eficiência *on-target* (cenário em que o par PegRNA-DNA é complementar por construção) não exercita suas linhas mais informativas. Analogamente, a Figura 5D do artigo já indicava a equivalência de desempenho entre o modelo com e sem DNABERT. Ambos os resultados negativos, longe de constituírem falhas, representam achados empíricos que quantificam a contribuição nula desses componentes, algo que o artigo não avalia de forma isolada.

3.3 Validação: três protocolos e a questão do vazamento

A Tabela 10 apresenta os três protocolos de validação aplicados ao modelo de produção (Fase 2, 14 *features*), que caracterizam aspectos distintos da capacidade preditiva.

Tabela 10: Protocolos de validação do modelo de 14 *features*.

Protocolo	Pearson	Spearman	Aspecto medido
5-fold por linha	$0,89 \pm 0,01$	$0,91 \pm 0,01$	Interpolação intra-alvo
HT-Test (<i>holdout</i>)	0,74	0,78	Generalização (vazamento parcial)
GroupKFold por alvo	$0,67 \pm 0,01$	$0,72 \pm 0,01$	Generalização real

A progressão decrescente do coeficiente de Pearson — de 0,89 no particionamento por linha para 0,74 no *holdout* e 0,67 no particionamento por alvo — não decorre de instabilidade do ajuste, mas do grau de independência entre treino e teste que cada protocolo efetivamente assegura. A caracterização do conjunto de dados explica a magnitude dessa progressão: os 43.149 registros distribuem-se por apenas 1.956 alvos distintos, com média superior a vinte extensões de pegRNA por alvo. Sob particionamento aleatório por registro, portanto, praticamente todo alvo presente na partição de teste comparece também na de treino, ainda que representado por outras extensões, e o modelo é avaliado sobre uma tarefa de interpolação em contexto genômico já observado. O protocolo GroupKFold, ao manter íntegro cada alvo dentro de uma única partição, elimina esse compartilhamento e mede a generalização a *loci* inéditos ($r = 0,67$). A diferença de 0,22 entre os dois extremos quantifica, assim, a parcela do desempenho aparente atribuível ao vazamento estrutural.

Implicação sobre o artigo de referência

O artigo emprega validação cruzada particionada por linha e reporta desempenho da ordem de 0,80. Como essa metodologia está sujeita ao mesmo vazamento estrutural aqui caracterizado, é plausível que a generalização real do modelo original a *loci* inéditos seja substancialmente inferior ao valor reportado. O presente trabalho, ao reportar os três protocolos, oferece uma avaliação mais rigorosa que a do artigo original.

3.4 Sobre a legitimidade dos métodos de validação cruzada

A caracterização do vazamento estrutural, apresentada na Seção 2.7 e quantificada acima, não deve ser interpretada como uma desqualificação incondicional do protocolo particionado por registro. A pertinência de cada protocolo depende, de forma decisiva, do cenário de uso ao qual a ferramenta se destina — e existe um cenário legítimo, de relevância prática, para o qual a avaliação com alvos compartilhados constitui a medida correta, e não um artefato de vazamento.

Consideram-se dois modos de utilização distintos da ferramenta em produção. No primeiro, o usuário busca desenhar pegRNAs para um *locus* genômico inédito, jamais caracterizado experimentalmente; nesse caso, o modelo deve generalizar para um alvo ausente do conjunto de treino, e a métrica operacionalmente relevante é a fornecida pelo protocolo **GroupKFold** particionado por alvo ($r = 0,67$). No segundo modo, o usuário busca otimizar o conjunto de pegRNAs para um *locus* já caracterizado (presente no conjunto de treino com um subconjunto de suas extensões possíveis), procurando, entre as demais extensões, aquelas de maior eficiência predita. Nesse segundo cenário, a tarefa do modelo é precisamente a interpolação intra-alvo: prever a eficiência de novas extensões de um alvo cujo perfil já foi parcialmente observado.

Legitimidade do cenário de uso referente ao vazamento parcial

Para o segundo modo de utilização, a presença do alvo no conjunto de treino não configura vazamento espúrio, mas a reprodução fiel da condição de produção. Um pesquisador que já dispõe de dados experimentais para um *locus* e deseja selecionar extensões adicionais de maior eficiência opera exatamente sob a hipótese que o protocolo particionado por registro avalia. Sob essa ótica, o coeficiente de 0,89 não é uma superestimação enganosa, mas a métrica pertinente ao caso de uso de otimização intra-alvo — assim como o coeficiente de 0,67 é a métrica pertinente ao caso de uso de generalização a *loci* inéditos.

Essa distinção justifica, retrospectivamente, a decisão de preservar e reportar ambos os protocolos, em vez de descartar o particionamento por registro como metodologicamente inadequado. Cada protocolo caracteriza um regime de uso real e distinto: o particionamento por alvo mede a descoberta em *loci* novos; o particionamento por registro mede a otimização em *loci* conhecidos. A configuração do conjunto de teste nativo (**HT-Test**), cujo vazamento é apenas parcial, situa-se de forma coerente entre os dois extremos, refletindo um cenário misto, no qual parte dos alvos de teste foi observada em treino e parte não, plausivelmente o mais representativo do uso agregado da ferramenta.

Cabe, todavia, uma ressalva de rigor. A legitimidade da avaliação com alvos compartilhados restringe-se ao segundo modo de utilização, e sua adoção como métrica única induziria a uma expectativa de desempenho excessivamente otimista para o cenário de descoberta. A prática metodologicamente honesta, e adotada neste trabalho, consiste em reportar os três protocolos de forma explícita, associando cada um ao regime de uso que efetivamente caracteriza, de modo que o usuário possa identificar a métrica pertinente à sua aplicação concreta.

3.5 Comparação com o artigo e impacto das adaptações

Sob a mesma metodologia do artigo (*k-fold* por linha), o modelo desenvolvido atinge Pearson 0,89 e Spearman 0,91, superando os valores reportados (0,80 e 0,77), e o faz sem a *encoding* $8 \times L$, sem o DNABERT e sem o pré-treino ClinVar. Do ponto de vista prático (ordenação de candidatos por eficiência), o coeficiente de Spearman de 0,78 no conjunto de *holdout* já supera o valor reportado no artigo, atendendo ao objetivo operacional da ferramenta.

O impacto das adaptações realizadas é, em síntese, positivo: a redução de 17 para 14 *features* melhora o desempenho; a reconstrução das *features* em inferência viabiliza a operação em sequências novas; e a caracterização do vazamento fornece uma medida honesta de generalização.

3.6 Trade-offs e fontes de erro

Trade-offs entre produção e completude

A tensão entre as duas frentes manifesta-se de modo concreto. A frente de produção favoreceria o encerramento na Fase 2, que fornece o melhor modelo. A frente de completude motivou a implementação integral das Fases 3 e 4, cujo resultado (a demonstração empírica de contribuição nula) possui valor metodológico próprio, ainda que não operacional. A arquitetura de versões alternáveis por configuração (*Forma B*) materializa esse equilíbrio, permitindo alternar entre o modelo de produção e o modelo de completude sem duplicação de código.

Possíveis fontes de erro

Identificam-se como fontes de erro remanescentes: a fidelidade de 93–99% (não integral) de algumas *features* MFE reconstruídas; a regra de canto do PAM no *encoding* $8 \times L$, inferida de exemplo único; e a decomposição do alinhamento de 73 posições, adotada como decisão de projeto na ausência de especificação do artigo. Nenhuma dessas fontes compromete o modelo de produção, que dispensa o *encoding* $8 \times L$.

3.7 Infraestrutura, custo computacional e *bottlenecks*

A execução do projeto foi condicionada, em diversos momentos, por restrições de *hardware*, de tempo de computação, de memória e de dependências de *software*. Esta subseção percorre essas dimensões, identificando os *bottlenecks* que efetivamente limitaram o ritmo do desenvolvimento e as decisões metodológicas deles decorrentes, servindo como uma referência para planejamentos de reproduções ou extensões futuras desse trabalho em ambientes dotados de capacidades de processamento distintas.

3.7.1 Hardware

O projeto foi conduzido na infraestrutura do *High Performance Computing Cluster* Heisenberg, disponível para uso aos discentes da Ilum - Escola de Ciência empregando-se inicialmente uma NVIDIA Tesla T4 de 16 GB de VRAM e, em fase posterior, de uma unidade de NVIDIA RTX 4090 com 24 GB de VRAM. A ausência de paralelismo de um mesmo treinamento entre múltiplas unidades disponíveis restringiu a estratégia de aceleração ao processamento de treinos independentes em unidades distintas. A introdução da unidade de maior capacidade constituiu um ponto de inflexão ao reduzir substancialmente o tempo de treino, por sua arquitetura mais recente e viabilizar a modalidade de *fine-tuning* do DNABERT (Fase 4b), inviável nos 16 GB das unidades T4 quando somado ao restante da arquitetura.

3.7.2 Tempo de computação

O tempo de treinamento constituiu o principal *bottleneck* do desenvolvimento, dado que um treino completo do modelo de produção, na unidade T4, demandava ~ 14 horas. Esse custo unitário elevado tornava cada iteração dispendiosa e motivou a adoção sistemática da distinção entre vias baratas e caras de avaliação, de modo que a avaliação por *holdout* (HT-Test), da ordem de segundos, foi aplicada para a medição de cada fase; enquanto a validação cruzada, manteve-se reservada à consolidação final.

Bottleneck: validação cruzada

A validação cruzada *k-fold*, ao exigir cinco treinos completos a partir do zero, projetava um custo da ordem de 70 horas em uma única unidade, constituindo o mais severo *bottleneck* do projeto. Sua mitigação apoiou-se em três estratégias: o pré-teste de curta duração, destinado a validar a mecânica da rotina antes de comprometer o custo integral; o salvamento incremental por *fold*, que preserva o progresso diante de interrupções; e, no protocolo sem vazamento, a execução de um subconjunto de *folds*, suficiente para confirmar a estabilidade da tendência observada.

3.7.3 Memória

O consumo de memória de vídeo constituiu o fator limitante da Fase 4. Em modo de inferência, a extração de *embeddings* do DNABERT revelou consumo reduzido, da ordem de fração de gigabyte, o que viabilizou trivialmente a abordagem congelada. O *fine-tuning*, entretanto, ao exigir a retenção de gradientes, dos estados do otimizador e das ativações intermediárias de todas as camadas do modelo BERT, multiplica esse consumo por fator substancial, situando-o em faixa incompatível com os 16 GB da unidade T4 quando somado ao restante da arquitetura. Essa restrição foi o determinante direto da postergação do *fine-tuning* até a disponibilização da unidade RTX 4090 com 24 GB, e

ilustra como a memória, e não apenas o tempo, condicionou a viabilidade de componentes arquiteturais para a execução do presente trabalho.

3.7.4 Pré-cálculo como estratégia de mitigação

A restrição de tempo motivou o pré-cálculo dos *embeddings* do DNABERT *frozen* durante a Fase 4a que, por serem, invariantes entre épocas, tais *embeddings* foram computados uma única vez e armazenados, eliminando a passagem pelo modelo BERT durante o treino. Essa estratégia converteu um custo proporcional ao número de épocas em um custo unitário, e exemplifica a mitigação de um *bottleneck* por meio de reorganização do fluxo, e não de aumento de recursos.

3.7.5 Dependências de *software*

O *bottleneck* de natureza não computacional mais relevante foi a compatibilidade das dependências legadas do DNABERT. A biblioteca **transformers**, em sua versão 2.5 utilizada no repositório original, impõe uma interface de programação distinta das versões atuais, na tokenização e no formato da saída do modelo, exigindo adaptações específicas no código de integração. A coexistência entre a biblioteca legada e versões modernas das demais dependências exigiu, ainda, atenção a incompatibilidades de interface, resolvidas caso a caso.

3.7.6 Síntese dos pontos de *bottlenecking*

A Tabela 11 sintetiza os principais *bottlenecks* identificados e as estratégias de mitigação adotadas.

Tabela 11: *Bottlenecks* do projeto e respectivas mitigações.

Dimensão	<i>Bottleneck</i>	Mitigação
Tempo	Treino unitário de ~14 h; validação cruzada de ~70 h	Instrumento barato para iteração; pré-teste de curta duração; salvamento incremental; subconjunto de <i>folds</i>
Memória	<i>Fine-tuning</i> do DNABERT incompatível com 16 GB	Postergação até a unidade de 24 GB; pré-cálculo na abordagem <i>frozen</i>
<i>Hardware</i>	Ausência de paralelismo intra-treino	Treinos independentes em unidades distintas
Dependências	Biblioteca legada e conflitos de versionamento	Adaptação da interface de programação e resolução caso a caso

A análise dos pontos de estrangulamento revela um padrão coerente com a filosofia do projeto: sempre que possível, os *bottlenecks* foram mitigados por reorganização do fluxo de

trabalho — pré-cálculo, avaliação em duas velocidades, salvamento incremental — antes de se recorrer ao aumento de recursos, reservado aos casos em que a reorganização se mostrava insuficiente, como no *fine-tuning* do DNABERT.

4 Validação Externa

4.1 Objetivo e escopo

As métricas apresentadas na Seção 3 são todas internas, ou seja, derivam de partições do mesmo conjunto de dados (**HT-training**) sobre o qual o modelo foi ajustado. Ainda que o protocolo **GroupKfold** por alvo elimine o vazamento estrutural, ele não elimina as idiosincrasias da biblioteca de origem, conservando o mesmo *design* experimental, plataforma de sequenciamento, faixa de comprimentos e normalização. Uma ferramenta destinada ao uso experimental requer, portanto, confronto com dados que não poderiam ter sido observados durante o ajuste: produzidos por outro laboratório, sob outro protocolo e em outra escala de medida.

Em vista disso, esta seção descreve essa validação externa. O experimento utilizado como referência é a varredura sistemática de comprimentos de PBS e RTT realizada por Sousa *et al.* [11] para a correção da mutação CFTR F508del, uma deleção de três nucleotídeos (CTT) responsável pela maioria dos casos de fibrose cística. No trabalho supracitado, 96 epegRNAs (*engineered prime-editing guide RNA*), correspondentes a todas as combinações de PBS $\in [10, 17]$ e RTT $\in \{30, \dots, 38, 41, 42, 43\}$ para um único *protospacer* (denominado NGG2), foram sintetizados e testados em células HEK293T, com a eficiência de correção quantificada por sequenciamento de alto rendimento.

Delimitações da validação

O objetivo é verificar se o ordenamento das combinações de comprimento predito pelo DTMP-Prime acompanha o ordenamento medido experimentalmente. Não se busca, contudo, concordância numérica absoluta.

Quanto à unidade da grandeza predita, cabe uma precisão. A saída do modelo herda a unidade da coluna **Measured PE efficiency** da Tabela S5, tomada como alvo de regressão sem qualquer transformação, e essa coluna é expressa em porcentagem de *reads* editados. Nesse sentido, a predição *não* está em unidades arbitrárias: é um ponto percentual. Ocorre que a camada de saída é uma projeção linear irrestrita, sem função de ativação limitadora, de modo que o valor predito não está confinado ao intervalo fisicamente admissível. A varredura aqui reportada evidencia esse fato de forma direta: 14 das 96 células da grade recebem predição *negativa*, o que é inadmissível para uma proporção de *reads*. A grandeza predita é, portanto, uma estimativa não calibrada em ponto percentual, e não uma probabilidade nem um

score normalizado.

Ainda assim, a comparação numérica direta permanece inadequada, por uma razão independente da unidade: o modelo foi treinado sobre uma biblioteca em sistema PE2, ao passo que a medida do artigo provém de um sistema PE4max com epegRNAs quimicamente modificados, em outro contexto cromatínico. As duas grandezas compartilham a unidade, mas não a escala. Compará-las por erro absoluto incorreria em erro de categoria; a comparação adequada é por posto.

Apesar disso, a escolha deste alvo específico não é conveniente, visto que o próprio artigo submeteu a mesma tarefa a duas ferramentas computacionais consolidadas, PRI-DICT 2.0 [7] e DeepPrime [13], reportando que nenhuma das duas incluiu a combinação vencedora (NGG2 PBS13 RTT41) entre suas 24 melhores sugestões. A recomendação final dos autores é que a triagem experimental não seja substituída pelos modelos atualmente disponíveis, tratando-se, portanto, de um caso reconhecidamente difícil, no qual o desempenho esperado *a priori* para qualquer preditor de eficiência é baixo.

4.2 Mapeamento do *design* experimental para o *pipeline*

É necessário destacar que o procedimento experimental de Sousa *et al.* consiste na correção da mutação ao partir da sequência portadora da F508del, na qual insere-se então a sequência CTT faltante. Na estrutura de *input ref/alt* adotada no DT-MP-Prime, isso significa que **ref** corresponde à sequência mutante e **alt** às sequência selvagem (*wild type*).

O *protospacer* NGG2 foi identificado por varredura de PAM NGG na fita reversa da região e confirmado por reconstrução: entre os cinco *protospacers* NGG da fita (–) presentes no fragmento de 189 pb, apenas TCTGTATCTATATTCATCAT (PAM AGG, sítio de corte na posição 143) gera, para PBS de 13 nt, a sequência ATGAATATAGATA publicada nos métodos do artigo. Todas as análises são restritas a esse *protospacer*, de modo a espelhar o painel experimental.

Relaxação de parâmetros

O *nick* especificado por NGG2 situa-se a 14 nt da posição da edição. O valor padrão de **max_target_to_sgRNA** no **config.yaml** é 10, e o mecanismo de relaxamento automático só é acionado quando nenhum *protospacer* válido é encontrado, o que não ocorre neste cenário, pois um *protospacer* da fita (+) satisfaz o critério. A validação então é executada sob a configuração **max_target_to_sgRNA = 30**.

A análise desenvolvida por Sousa *et al.* corresponde a PE4max, isto é, PE2 acrescido de MLH1dn, sem guia de nickagem. Em razão disso, a varredura é executada com **max_ngRNA_distance = 0**, o que força o *pipeline* a emitir a linha PE2 pura de cada par

PBS $\times$ RTT. Por completude, a condição PE3 (com ngRNA, agregada por máximo entre os quatro guias de nickagem encontrados) também é registrada.

4.3 Divergências metodológicas

A Tabela 12 consolida as diferenças entre o experimento de referência e as condições sob as quais o DTMP-Prime foi treinado, as quais não invalidam a comparação, mas delimitam o que ela pode aferir, de modo que três dessas divergências merecem tratamento detalhado, por serem as que determinam a interpretação dos resultados.

Tabela 12: Divergências entre o experimento de Sousa *et al.* (2024) e o domínio de treino do DTMP-Prime.

Dimensão	Sousa <i>et al.</i> (Fig. 2b)	Domínio de treino	Consequência
Sistema de edição	PE4max (PE2 + MLH1dn), epegRNA com motivo pseudonó 3'	PE2 com pegRNA canônico	Deslocamento de escala; ordenação preservada apenas sob hipótese de aditividade
Faixa de RTT	30–43 nt	≤ 20 nt	Extrapolação; o tokenizador trunca em 20 nt
Faixa de PBS	10–17 nt	≤ 17 nt	Dentro do domínio
Tipo de edição	Inserção de 3 nt (CTT)	Predominantemente substituições e indels curtos	Categoria representada, porém minoritária
Guia de nickagem	Ausente no painel	PE2 e PE3	Comparação feita em modo PE2
Grandeza medida	% de <i>reads</i> com inserção de CTT ($n = 3$ réplicas)	Ponto percentual, não calibrado e sem limites	Comparação apenas por posto
Faixa dinâmica	0,005–0,73%	Ampla	Impõe teto à correlação alcançável
Contexto	<i>Locus CFTR</i> em HEK293T, cromatina desfavorável	Bibliotecas integradas	Efeito comum a toda a grade; não altera a ordenação interna

Extrapolação no eixo do RTT

A biblioteca de treino cobre RTT de até 20 nt, e o tokenizador de ordem 3 (presente no `script tokenizer_order3.py`, constante `MAX_RT = 20`) trunca sequências mais longas. Toda a grade do artigo situa-se fora desse domínio. A inspeção direta das sequências geradas permite quantificar o efeito desse truncamento (Tabela 13): a janela de 20 nt efetivamente tokenizada desliza conforme o comprimento do RTT solicitado e, a partir de RTT = 35 nt, deixa de conter a edição a ser instalada. Em dez das quatorze combinações

de comprimento avaliadas, portanto, o ramo sequencial do modelo processa um trecho do molde no qual a inserção de CTT está ausente.

Tabela 13: Janela de 20 nt efetivamente tokenizada do RTT (mostrada para $PBS = 13$), sob a geometria corrigida. A inserção corretiva de CTT é destacada. A partir de $RTT = 35$ nt a edição sai da janela.

RTT (nt)	Janela tokenizada (20 nt, 5' → 3')	Contém a edição?
30	AAGAAAATATCATCTTTGGT	sim
31	AAAGAAAATATCATCTTTGG	sim
32	TAAAGAAAATATCATCTTTG	sim
33	TTAAAGAAAATATCATCTTT	sim
34	ATTAAAGAAAATATCATCTT	sim
35	CATTAAAGAAAATATCATCT	não
36	CCATTAAAGAAAATATCATC	não
37	ACCATTAAAGAAAATATCAT	não
38	CACCATTAAAGAAAATATCA	não
41	TGGCACCATTAAAGAAAATA	não
42	CTGGCACCATTAAAGAAAAT	não
43	CCTGGCACCATTAAAGAAAA	não

Inexistência de RTT dentro do domínio

A grade complementar executada com $RTT \in [10, 20]$, prevista como controle de extrapolação, retornou zero candidatos para o *protospacer* NGG2 sob a geometria corrigida. O motivo é geométrico, pois com o *nick* a 14 nt da edição, o RTT precisa de ao menos 17 nt para conter a inserção e de 22 nt para satisfazer adicionalmente o requisito de 5 nt de homologia além da edição (`min_distance_RTT5`). Como o teto do domínio de treino é 20 nt, não existe comprimento de RTT simultaneamente válido para este alvo e contido no domínio de treino. A escolha do artigo por iniciar a varredura em 30 nt não é, portanto, arbitrária, e a extrapolação neste eixo é inevitável, não sendo um artefato do *design* da validação.

Faixa dinâmica estreita da referência

A eficiência experimental varia de 0,005% a 0,73% ao longo de toda a grade. Sousa *et al.* reconhecem que a estreita faixa dinâmica desses ensaios iniciais tornou algumas decisões de desenho — notadamente a seleção de comprimentos de PBS e RTT — substancialmente subjetivas, e recomendam, para trabalhos futuros que enfrentem eficiências iniciais baixas, ampliar a faixa dinâmica antes de decidir. Parte da matriz de referência é, portanto, ruído de medição, o que impõe um limite à correlação máxima atingível por qualquer preditor.

4.4 Protocolo implementado

A validação foi implementada no módulo `validation/`, executável em ambiente HPC por submissão SLURM, e organiza-se em três etapas.

Etapa 1 — Controle positivo de geometria

A comparação de eficiências pressupõe que o *pipeline* seja capaz de construir as moléculas efetivamente testadas no experimento de referência. Essa premissa é verificada antes de qualquer análise quantitativa, por meio de um controle positivo: solicita-se ao *pipeline* a construção do epegRNA canônico do artigo (*protospacer* NGG2, PBS de 13 nt, RTT de 41 nt) e confrontam-se as sequências obtidas com as publicadas nos métodos, na variante SE0, isto é, sem as edições silenciosas, que o DTMP-Prime não projeta. O critério de aprovação é a identidade caractere a caractere.

A necessidade desse controle decorre da estrutura do argumento que a validação pretende sustentar. Sem ele, os dois desfechos possíveis da comparação seriam igualmente inconclusivos: uma correlação favorável obtida a partir de sequências divergentes das testadas não constituiria evidência de capacidade preditiva, e uma correlação desfavorável não poderia ser atribuída ao modelo, e sim à construção incorreta das moléculas avaliadas. O controle positivo é, portanto, condição de interpretabilidade do resultado, e não uma verificação acessória.

Divergência geométrica identificada pelo controle

A aplicação do controle revelou que a implementação de `sgRNA.find_PBS` extrai a região do PBS do mesmo lado do *nick* que o RTT, em ambas as fitas, de modo que a região atribuída ao PBS coincide, em coordenadas genômicas, com um trecho da região do RTT. Biologicamente, o PBS hibridiza com a extremidade 3' da fita nicada, situada do lado oposto ao molde de transcrição reversa. Identificaram-se ainda, para *protospacers* da fita (–), duas divergências adicionais: a aplicação indevida de reverso-complemento ao RTT, cujo molde tem, nesse caso, o sentido da fita (+); e a leitura do RTT sobre a sequência `alt` com o índice de corte calculado sobre a sequência `ref`, o que desloca a âncora por $|\text{alt}| - |\text{ref}|$ nucleotídeos quando a mutação é um indel a montante do corte, situação que corresponde exatamente ao caso da inserção de CTT.

A correção geométrica foi implementada em `validation/geometry_fix.py`, por substituição dos dois métodos em tempo de execução, sem alteração do repositório. O controle é executado sob as duas convenções — `asis`, a vigente no repositório, e `fixed`, a corrigida — e o resultado consta da Tabela 14: apenas a geometria corrigida reproduz o epegRNA publicado, e o faz integralmente, tanto no PBS quanto no RTT. Toda a varredura descrita adiante é executada exclusivamente sob a convenção corrigida.

Tabela 14: Controle positivo de geometria: reconstrução do epegRNA canônico NGG2 PBS13 RTT41 publicado no artigo (versão SE0, sem edições silenciosas). O critério é a coincidência caractere a caractere.

Geometria	PBS reproduzido	RTT reproduzido
asis	não	não
fixed	sim	sim

Etapa 2 — Varredura e registro integral

Sob a geometria corrigida, e nos modos PE2 e PE3, o *pipeline* é executado sobre o fragmento de 189 pb do *locus CFTR*, com $PBS \in [10, 17]$ e $RTT \in [30, 43]$. A busca identificou três *protospacers* válidos, dos quais o NGG2 é o único correspondente ao painel experimental; para ele, foram gerados 112 candidatos em modo PE2 e 448 em modo PE3, estes últimos distribuídos entre quatro guias de nickagem. O conjunto completo de candidatos, somados os três *protospacers*, compreende 296 registros em PE2 e 1.368 em PE3.

Registram-se, sem filtragem: a matriz **rawX** completa, a matriz numérica de *features*, o inventário de *protospacers* com suas distâncias ao alvo, o recorte do *protospacer* NGG2 e as matrizes agregadas por máximo e por média. Um manifesto em formato JSON preserva as versões de biblioteca, a disponibilidade de GPU, o identificador de *commit* do repositório e a totalidade dos parâmetros efetivos, de modo a permitir a reprodução exata da execução.

Feature DeepSpCas9 indisponível na execução reportada

Na execução aqui reportada, a chamada a `get_DeepSpCas9_score` falhou para todos os *protospacers*, com o erro `AttributeError: module 'numpy' has no attribute 'bool8'`. A causa é uma incompatibilidade entre a versão da biblioteca `genet` (0.17.2) e a do `numpy` (2.0.1) presente no ambiente, uma vez que o alias `np.bool8` foi removido a partir do `numpy` 2.0. O *pipeline* trata a falha por substituição silenciosa do `score` por 0,0, de modo que a *feature* DeepSpCas9 — uma das 14 *features* escalares — entrou constante e nula na inferência.

Como todas as células da grade compartilham o mesmo *protospacer*, essa *feature* seria constante de qualquer modo, e sua anulação não introduz variação espúria ao longo dos eixos de PBS e RTT. Não se trata, contudo, de efeito nulo: sendo o modelo não linear, o valor assumido por uma entrada constante desloca a superfície de resposta sobre as demais. Registra-se, portanto, que os valores absolutos aqui reportados foram obtidos sob essa condição degradada, e que a correção da incompatibilidade é prioridade imediata — não apenas para a validação, mas para o uso em produção, no qual a mesma falha reduziria silenciosamente a 14 *features* efetivas a 13.

Etapa 3 — Comparação

A matriz predita é confrontada com a matriz experimental por meio dos coeficientes de Pearson, Spearman e Kendall, acompanhados de intervalos de confiança de Fisher e de um teste de permutação com 20.000 reamostragens (que não pressupõe normalidade e é apropriado ao tamanho amostral de $n = 96$). Reportam-se ainda as correlações dos perfis marginais de PBS e de RTT, as correlações internas a cada linha e a cada coluna, e a concordância no extremo superior da distribuição.

Sobre a matriz experimental de referência

Os valores experimentais foram obtidos por digitalização do painel publicado, mapeando a cor mediana de cada célula contra a barra de cores por vizinho mais próximo no espaço RGB, em rotina auditável (`validation/digitize_fig2b.py`). O controle de qualidade é o próprio texto do artigo, que declara média de 0,61% para a combinação NGG2 PBS13 RTT41; a digitalização recupera 0,62% para essa célula. O erro de quantização resultante, da ordem de $\pm 0,02$ ponto percentual, é adequado a uma comparação por posto e inadequado a uma comparação numérica absoluta, que de todo modo não é o objetivo. A rotina de análise aceita, alternativamente, o arquivo *Source Data* oficial do artigo, caso venha a ser obtido.

4.5 Resultados

As tabelas e o mapa de calor desta subseção são gerados automaticamente pela rotina de análise do módulo de validação e incorporados ao documento por um fragmento dedicado, de modo que qualquer reexecução da validação os atualiza sem intervenção manual.

Reporta-se uma única condição, doravante designada **modo PE2**: geometria verificada contra o epegRNA publicado e ausência de guia de nickagem, espelhando o painel PE4max original. A condição PE3, também executada, é discutida a seguir por motivo distinto do desempenho.

Tabela 15: Concordância entre a eficiência predita pelo DTMP-Prime e a eficiência de correção da *CFTR* F508del medida por Sousa *et al.* (2024, Fig. 2b), sobre a grade completa PBS $\times$ RTT do *protospacer* NGG2. Condição de referência: geometria verificada contra o epegRNA publicado, sem guia de nickagem.

Coefficiente	Valor	IC _{95%} / p
Pearson r	-0,120	[-0,313; 0,083]
Spearman ρ	-0,152	[-0,342; 0,050]
Kendall τ	-0,103	$p = 0,142$
n de células comparadas		96
p do teste de permutação (Spearman)		0,139

Tabela 16: Correlação dos perfis marginais na condição de referência: médias por comprimento de PBS (agregando RTT) e por comprimento de RTT (agregando PBS). O perfil de PBS está integralmente dentro do domínio de treino; o de RTT, integralmente fora dele.

Perfil	n	Pearson	Spearman
marginal PBS	8	-0,203	-0,167
marginal RTT	12	0,016	-0,322

Tabela 17: Concordância no extremo superior da distribuição, na condição de referência: sobreposição entre os k melhores candidatos experimentais e preditos, e posto predito das combinações destacadas no artigo.

Medida	Valor
Sobreposição entre os 5 melhores	0/5
Sobreposição entre os 10 melhores	1/10
Posto predito de PBS13 RTT41 (adotada no artigo)	69 de 96
Posto predito de PBS14 RTT41 (ótimo experimental)	50 de 96

Irrelevância do guia de nickagem para o modelo

As condições PE2 e PE3 produzem previsões numericamente idênticas: a diferença máxima entre as duas matrizes, sobre as 96 células, é exatamente nula. O resultado é estrutural, dado que nem as 14 *features* escalares nem os nove tensores de *tokens* codificam qualquer informação sobre o ngRNA. O modelo de produção da Fase 2, portanto, não distingue PE2 de PE3. A classificação automática entre PE2 e PE3 exposta na interface é, consequentemente, uma anotação do candidato, e não um fator que module o *score* predito.

Ausência de correlação sobre a grade completa

Sobre as 96 células da grade, a correlação entre predição e medida experimental é nula: $\rho = -0,152$, com $p = 0,139$ pelo teste de permutação e intervalo de confiança de 95% igual a $[-0,342; 0,050]$, que contém o zero. Os coeficientes de Pearson ($r = -0,120$) e de Kendall ($\tau = -0,103$) conduzem à mesma leitura.

A concordância no extremo superior da distribuição, que é o regime de interesse prático para uma ferramenta de priorização, é igualmente ausente. A combinação identificada experimentalmente como ótima (PBS14 RTT41) ocupa o posto 50 de 96 na predição, e a efetivamente adotada pelo artigo (PBS13 RTT41), o posto 69 de 96. Não há sobreposição alguma entre os cinco melhores candidatos experimentais e os cinco melhores preditos, e a sobreposição entre os dez melhores é de um único par.

A predição concentra seus valores mais altos na coluna de RTT = 30 nt, combinada aos maiores comprimentos de PBS: o candidato de maior escore é PBS17 RTT30, ao qual corresponde eficiência experimental de 0,005%, o mínimo de toda a matriz de referên-

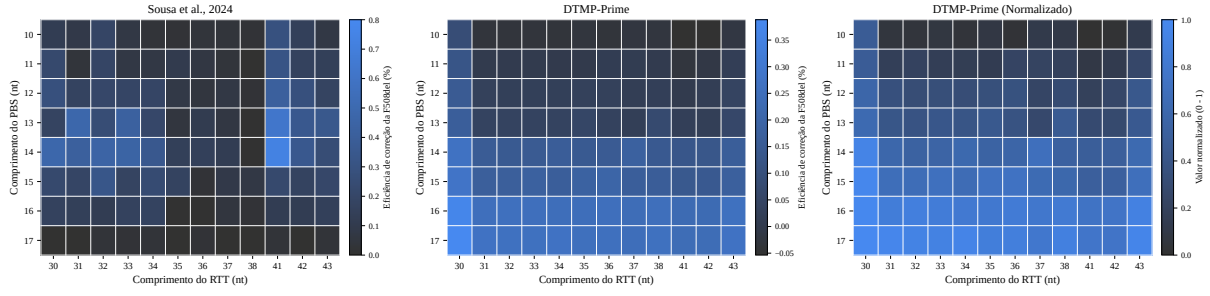


Figura 1: Varredura PBS $\times$ RTT para o *protospacer* NGG2. À esquerda, a matriz experimental de Sousa *et al.* (2024); ao centro, a eficiência predita pelo DTMP-Prime na mesma grade; à direita, a predição convertida em percentil, que é a escala em que a comparação é legítima. A paleta reproduz a do painel original.

cia. A ordenação predita e a medida não apenas divergem, portanto, como situam seus respectivos ótimos em regiões opostas da grade.

Reprodução do ramo ascendente do perfil de PBS

A decomposição do perfil marginal de PBS (Figura 2) é mais informativa que o coeficiente global. A curva experimental é unimodal: cresce de PBS 10 a PBS 14 e colapsa em seguida, com PBS 17 praticamente abolindo a edição. A predição do DTMP-Prime é monotonicamente crescente em todo o intervalo. A consequência é quantificada na Tabela 18: no ramo ascendente ($10 \leq \text{PBS} \leq 14$), a concordância de posto é perfeita ($\rho = 1,000$; Pearson = 0,939); no ramo descendente ($14 \leq \text{PBS} \leq 17$), é perfeitamente invertida ($\rho = -1,000$; Pearson = $-0,913$). O coeficiente calculado sobre o intervalo completo, $\rho = -0,167$, resulta da média desses dois regimes opostos e oculta a estrutura de ambos.

Tabela 18: Decomposição do perfil marginal de PBS por ramo da curva experimental (modo PE2). Valores calculados a partir do arquivo `analise/matriz_predita_fixed_pe2.csv` da execução reportada.

Intervalo de PBS	Spearman	Pearson	Regime da curva experimental
$10 \leq \text{PBS} \leq 14$	1,000	0,939	Ramo ascendente
$14 \leq \text{PBS} \leq 17$	$-1,000$	$-0,913$	Ramo descendente (colapso)
$10 \leq \text{PBS} \leq 17$	$-0,167$	$-0,203$	Intervalo completo

Esse padrão é a assinatura característica da extrapolação monotônica. Dentro do domínio de treino, comprimentos maiores de PBS estão associados a maior temperatura de fusão do híbrido e, em média, a maior eficiência, de forma que essa tendência capturada pelo modelo é estendida indefinidamente. A queda experimental observada em $\text{PBS} \geq 15$ não é capturada. Sousa *et al.* atribuem as baixas eficiências observadas nesse regime à instabilidade do conjunto PBS–RTT do epegRNA e a interações em *cis* entre o espaçador e o PBS [11]; a existência de um comprimento ótimo intermediário de PBS, tipicamente

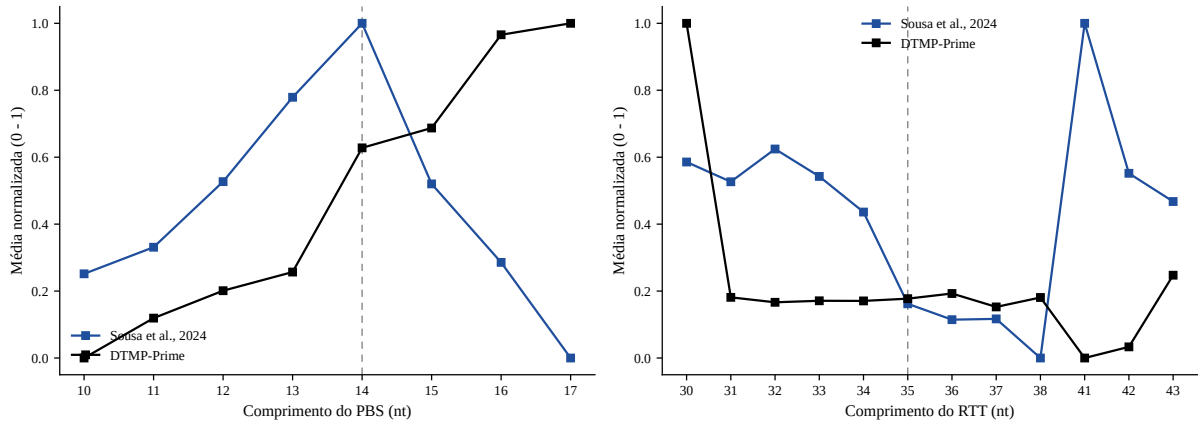


Figura 2: Perfis marginais normalizados, obtidos pela média de cada linha (painel esquerdo) e de cada coluna (painel direito) das matrizes da Figura 1. **À esquerda**, a média por comprimento de PBS: a curva experimental é unimodal, ao passo que a predição cresce monotonicamente em todo o intervalo. A linha vertical tracejada assinala PBS = 14 nt, máximo experimental, que separa o ramo ascendente do descendente; a decomposição da concordância por ramo consta da Tabela 18. **À direita**, a média por comprimento de RTT. A linha vertical tracejada assinala RTT = 35 nt, comprimento a partir do qual a janela de 20 nt efetivamente tokenizada deixa de conter a inserção de CTT (Tabela 13). O eixo horizontal do painel direito é categórico: os comprimentos 39 e 40 não constam do painel publicado.

próximo de 13 nt, já havia sido reportada na descrição original do *prime editing* [2]. Trata-se, em qualquer das hipóteses, de um efeito não monotônico, que o conjunto de *features* disponível ao modelo não é capaz de representar. O achado é coerente com a constatação da Seção 3 de que apenas as *features* escalares carregam sinal preditivo: sendo a temperatura de fusão e o conteúdo GC funções monotônicas do comprimento, um modelo que se apoia predominantemente nelas herda essa monotonicidade.

Domínio do eixo do RTT pelo artefato de truncamento

O perfil marginal de RTT predito é essencialmente plano (Figura 2, painel direito). Exce- tuando RTT = 30 nt, que recebe valor destacadamente superior, as demais onze colunas concentram-se em uma faixa estreita, entre 0,086 e 0,122 em média, ao passo que a medida experimental varia por um fator de dezesseis ao longo do mesmo eixo.

Esse comportamento decorre diretamente do truncamento documentado na Tabela 13. A janela de 20 nt efetivamente tokenizada desliza conforme o comprimento solicitado e, de RTT = 35 nt em diante, deixa de conter a inserção de CTT. O que varia ao longo do eixo não é, portanto, uma representação do molde de transcrição reversa em sua extensão, mas apenas qual trecho de 20 nt do flanco a montante foi capturado pela janela; nas dez colunas mais longas, esse trecho sequer inclui a edição a ser instalada. A descontinuidade entre RTT = 30 nt e as demais colunas é coerente com essa leitura: é na primeira coluna que a janela contém a edição em posição mais central.

O eixo do RTT não constitui, por conseguinte, predição informada sobre o compri-

mento do molde, e a correlação nula ali observada ($\rho = -0,322$; $p = 0,319$) mede o limite de representação do tokenizador, não a capacidade preditiva do modelo.

Síntese

Resultado da validação externa

No único regime em que a comparação é metodologicamente sólida — o eixo do PBS, integralmente dentro do domínio de treino, sob geometria verificada contra o epegRNA publicado — o DTMP-Prime reproduz a direção do efeito no ramo ascendente da curva experimental ($\rho = 1,000$ para $10 \leq \text{PBS} \leq 14$) e falha em identificar o ótimo, extrapolando monotonicamente para além dele. Sobre a grade completa, incluindo o eixo extrapolado do RTT, não há correlação com a medida experimental ($\rho = -0,152$; $p = 0,139$), e a concordância no extremo superior da distribuição é nula.

Este resultado é consistente com o que o próprio artigo de referência relata para PRIDICT 2.0 e DeepPrime no mesmo alvo, e não contradiz as métricas internas da Seção 3: mede um regime — extrapolação de comprimento em *locus* inédito, sob outro sistema de edição — que nenhuma daquelas métricas cobre.

4.6 Limitações e alcance da conclusão

Registram-se explicitamente as seguintes limitações, sob pena de superestimar — ou de sobreinterpretar negativamente — o que este exercício estabelece.

1. **Locus e protospacer únicos:** A validação cobre 96 combinações de comprimento de um só alvo. Ela não estima desempenho médio sobre *loci* arbitrários; para isso, a métrica pertinente continua sendo o protocolo **GroupKfold** da Seção 3. Um resultado negativo em $n = 1$ alvo não estabelece falha sistemática, assim como um positivo não estabeleceria competência.
2. **Inevitabilidade de extrapolação no eixo do RTT:** Conforme demonstrado, não existe comprimento de RTT simultaneamente válido para este alvo e contido no domínio de treino. A ausência de correlação nesse eixo mede o limite do domínio de aplicabilidade do modelo, não um defeito de ajuste.
3. **Ruído não desprezível da referência:** Com valores entre 0,005% e 0,73% e $n = 3$ réplicas biológicas, parte da variação da matriz experimental é erro de medida. A correlação máxima atingível por qualquer preditor é inferior a 1, em magnitude desconhecida na ausência dos desvios-padrão célula a célula.
4. **Influências sistemáticas intrínsecas:** Diferenças de sistema (PE4max *vs.* PE2), de modificação química do guia e de contexto cromatínico afetam a magnitude absoluta.

O argumento de que não afetam a ordenação interna da grade é uma hipótese de aditividade, plausível mas não demonstrada. O próprio artigo oferece evidência favorável a ela: a retriagem de comprimentos na presença de todos os aprimoramentos (Extended Data Fig. 7a) confirmou que PBS13 RTT41 permanecia entre as combinações ótimas.

5. **Feature degradada na execução reportada:** Conforme a Etapa 2, o escore DeepSpCas9 entrou nulo por incompatibilidade de versões. Sendo constante ao longo de toda a grade, a anulação não produz variação espúria entre células, mas desloca a superfície de resposta do modelo e afeta os valores absolutos reportados. A execução deve ser repetida após a correção da dependência, ainda que não se espere reversão da conclusão qualitativa.
6. **Matriz de referência digitalizada:** Enquanto o *Source Data* oficial não for incorporado, os valores experimentais carregam erro de quantização adicional. Dada a magnitude do efeito observado — ausência de correlação, e não correlação marginal —, é implausível que a substituição da matriz altere a conclusão.
7. **Ausência de contrafactual negativo:** O desenho não inclui um alvo para o qual se saiba que o modelo deveria acertar, o que permitiria distinguir limitação do modelo de limitação do protocolo de comparação. Uma extensão natural seria replicar o protocolo para os painéis PE2 da Figura 1b e das Extended Data Figs. 2 e 3 do mesmo artigo, que cobrem sete *protospacers* adicionais.

O que a validação factualmente estabelece

Sob as condições declaradas, este exercício estabelece: para o *locus CFTR* F508del e o *protospacer* NGG2, a eficiência predita pelo DTMP-Prime não recupera a ordenação experimental sobre a grade PBS × RTT como um todo, recupera perfeitamente a ordenação no ramo ascendente do perfil de PBS dentro do domínio de treino, e não identifica o ótimo de PBS, extrapolando monotonicamente para além dele. Qualquer generalização além desse enunciado — para outros alvos, outras faixas de comprimento ou outros sistemas de *prime editing* — não é sustentada pelos dados aqui apresentados.

4.7 Implicações para o trabalho

A validação externa produz quatro consequências práticas, de naturezas distintas.

Sobre o alcance da divergência geométrica

Cabe delimitar com precisão o alcance da divergência identificada pelo controle positivo, uma vez que ele determina o custo de sua correção.

O caminho de treino não é afetado. As funções `find_PBS` e `find_RTT` são invocadas em um único ponto do repositório, no procedimento de busca de *protospacers*, pertencente

exclusivamente ao *pipeline* de inferência. O treino obtém as sequências diretamente da coluna de extensão 3' do pegRNA da Tabela S5, particionada pelos comprimentos tabelados, e as 14 *features* escalares de colunas da mesma planilha, das quais também derivam as estatísticas de normalização. Consequentemente, os pesos treinados, os arquivos de normalização, as métricas dos três protocolos da Seção 3 e as conclusões sobre as Fases 3 e 4 permanecem válidos, e nenhum retreino é necessário.

O caminho de inferência é afetado integralmente. São atingidos todos os valores de PBS e RTT já emitidos e, por consequência, doze das quatorze *features* escalares — escapam apenas a energia de enovelamento que depende somente do espaçador e o escore DeepSpCas9 —, os nove tensores de *tokens* e os atributos derivados de comprimento e composição. O erro de lado do PBS não se restringe a uma das fitas: na fita (+) o PBS emitido corresponde ao sufixo da janela do RTT e, na fita (−), ao seu prefixo, de modo que o espaço de busca é atingido em sua totalidade. Específicas da fita (−) são apenas as duas divergências adicionais.

Disso decorre uma observação relevante: configura-se um descasamento entre treino e inferência. O modelo foi ajustado sobre PBS e RTT geometricamente corretos e recebe, em produção, sequências que não o são. A correção, portanto, alinha a inferência ao regime de treino, e não há razão para supor que os pesos tenham se acomodado ao defeito, uma vez que jamais o observaram. Essa constatação também sustenta a interpretação do resultado desta seção: a varredura foi executada sob a única convenção em que modelo e entrada operam sob a mesma geometria, o que impede atribuir o resultado negativo à correção aplicada.

Sobre a integridade do vetor de *features*

A falha silenciosa da *feature* DeepSpCas9, descrita na Etapa 2, não é específica da validação. Trata-se de uma incompatibilidade de versões que se manifesta em qualquer execução do *pipeline* no mesmo ambiente, inclusive pela interface, e cujo único sinal é uma mensagem de aviso no fluxo de saída padrão. O comportamento de degradação silenciosa é inadequado para uma *feature* de entrada do modelo: recomenda-se que a indisponibilidade do escore interrompa a execução ou, no mínimo, seja propagada ao usuário junto aos resultados, e que a versão de `numpy` seja fixada no arquivo de dependências.

Sobre a interface

Três comportamentos merecem revisão. O primeiro é o descarte silencioso de *protospacers* cuja distância à edição excede `max_target_to_sgRNA`, que suprimiu, sob configuração padrão, justamente o *protospacer* adotado pelo artigo de referência, e que deve ao menos gerar aviso. O segundo é a exibição da classificação PE2/PE3, que não deve sugerir que o escore predito considere o guia de nickagem, uma vez que demonstradamente não o considera. O terceiro diz respeito à apresentação do próprio escore: a documentação de auxílio descreve-o como valor no intervalo $[0, 1]$, ao passo que a saída não é normalizada e admite

valores negativos e superiores à unidade, de modo que a formatação numérica, os limites de cor e a escala das barras de progresso requerem recalibração à faixa efetivamente observada.

Sobre o escopo declarado da ferramenta

O resultado recomenda que a documentação destinada ao usuário final explicita o domínio de aplicabilidade do modelo — PBS de até 17 nt e RTT de até 20 nt —, com advertência quanto à extrapolação fora dessa faixa. Mais importante, corrobora a recomendação dos próprios autores do artigo de referência, de que a triagem experimental não seja substituída por predição computacional, e situa o DTMP-Prime como instrumento de priorização de candidatos para triagem, e não de seleção de candidato único.

5 Conclusão

5.1 Síntese dos objetivos alcançados

O projeto reconstruiu, validou e analisou criticamente o modelo DTMP-Prime, atingindo os objetivos propostos. A implementação corrigiu os defeitos estruturais do repositório original, reconstruiu cada componente arquitetural em fases individualmente validáveis, quantificou a contribuição isolada de cada camada e estabeleceu um conjunto de protocolos de validação capaz de distinguir a interpolação intra-alvo da generalização real a novos *loci* genômicos. O modelo de produção resultante iguala ou supera o artigo de referência na mesma metodologia de avaliação, dispensando três dos componentes arquiteturais originais. A esse quadro somou-se, ao final, uma validação externa contra dados laboratoriais independentes (Seção 4), que delimita o domínio de aplicabilidade do modelo e evidencia limitações não detectáveis unicamente a partir das métricas internas.

5.2 Principais contribuições

- **Reconstrução fiel e validada** de todos os componentes arquiteturais do DTMP-Prime, com validação célula a célula contra gabaritos do artigo.
- **Quantificação da contribuição por camada**, demonstrando empiricamente que apenas as *features* agregam sinal preditivo — o que o artigo não avalia isoladamente — e que a degradação introduzida pelos demais componentes é proporcional à capacidade treinável excedente que adicionam.
- **Caracterização completa da generalização** por meio de três protocolos de validação, cada um associado a um cenário de uso distinto: a otimização de *pegRNAs* em *loci* já caracterizados (interpolação intra-alvo) e a descoberta em *loci* inéditos (generaliza-

ção real). A introdução do protocolo particionado por alvo, não reportado no artigo original, fornece uma medida de generalização anteriormente ausente.

- **Documentação sistemática das divergências** entre a descrição do artigo e a implementação, o uso da representação sequencial e a convenção de sinais da codificação, estabelecendo a primazia da evidência empírica sobre a descrição textual como princípio metodológico.
- **Arquitetura de versões alternáveis** entre produção e completude por configuração, sem duplicação de código.
- **Validação externa contra dados independentes**, confrontando as predições com a varredura sistemática de comprimentos de PBS e RTT de Sousa *et al.* [11] para a correção da *CFTR* F508del. O protocolo inclui um controle positivo de geometria que precede e condiciona qualquer comparação de eficiência, e que revelou uma divergência geométrica na construção do PBS e do RTT não detectável por métricas agregadas.

5.3 Limitações da implementação

Persistem as seguintes limitações, documentadas ao longo do trabalho: a não reprodutibilidade dos atributos $Tm2$, $Tm3$ e ΔTm em inferência; a fidelidade não integral de alguns atributos MFE reconstruídos; a ausência de especificação, no artigo, do alinhamento de 73 posições e da arquitetura do ramo convolucional, supridas por decisões de projeto documentadas; e a irreprodutibilidade prática do pré-treino em ClinVar (Fase 0), decorrente da escala dos dados, de sua indisponibilidade e da subespecificação do método. Nenhuma dessas limitações compromete o modelo de produção, que dispensa o *encoding* $8 \times L$ e a camada de *embedding* DNABERT.

A validação externa acrescentou quatro limitações de natureza distinta, por dizerem respeito ao comportamento do modelo em produção e não à fidelidade da reprodução. Primeiro, o modelo de produção é estruturalmente indiferente ao ngRNA, visto que nem as *features* escalares nem os tensores de *tokens* codificam informação sobre o ngRNA, de modo que as predições PE2 e PE3 são numericamente idênticas. Segundo, o tokenizador trunca o RTT em 20 nt, o que torna a predição não informada quanto ao comprimento do RTT fora dessa faixa. Terceiro, e mais consequente, o modelo extrapola monotonicamente o efeito do comprimento do PBS, reproduzindo o ramo ascendente da curva experimental mas não o seu ótimo — comportamento coerente com o achado central de que apenas as *features* escalares, que são funções monotônicas do comprimento, carregam sinal preditivo. Quarto, a grandeza predita não é calibrada: a camada de saída é uma projeção linear irrestrita, de modo que o valor emitido, embora herde a unidade percentual do alvo de treino, admite valores negativos e superiores à unidade, o que exige revisão da forma como é apresentado ao usuário. A essas soma-se a divergência geométrica na construção

do PBS e do RTT, restrita ao caminho de inferência — os pesos treinados e as métricas internas derivam das colunas tabeladas da Tabela S5 e permanecem válidos, dispensando retreino —, cuja correção está implementada e verificada, mas cuja incorporação ao código de produção implica reavaliar os resultados anteriormente produzidos pela interface.

5.4 Trabalhos futuros e possíveis extensões

Versão factível do pré-treino em ClinVar

Embora o pré-treino original seja irreproduzível em escala, delineou-se uma versão factível, baseada na subamostragem de variantes patogênicas do ClinVar, na geração dos *pegRNAs* candidatos correspondentes, no pré-treino auto-supervisionado por modelagem de linguagem mascarada e no posterior ajuste fino com transferência de aprendizado. Tal extensão deve ser desenvolvida com a arquitetura já consolidada, executando-se em primeiro lugar no *pipeline* final.

Modelo de arquitetura completa

Para fins de completude documental, pode-se treinar um modelo integrando simultaneamente todos os componentes (*features*, *encoding* e DNABERT), não para a melhoria da métrica, já saturada pela Fase 2, mas para registrar o desempenho da reprodução arquitetural integral do artigo.

Extensões operacionais

Do ponto de vista de produção, vislumbram-se a incorporação de *mixed precision* para a aceleração do treino, a classificação automática entre os modos PE2 e PE3, a implementação de testes automatizados de integração e de validação de entrada, e a condução de um piloto de usabilidade com usuário final da área biológica, com vistas a aferir a acessibilidade da ferramenta a pesquisadores sem formação computacional.

5.5 Considerações finais

O trabalho demonstra que a reprodução rigorosa de um modelo publicado transcende a mera reimplementação: envolve a identificação de divergências, a quantificação crítica da contribuição de cada componente e o estabelecimento de protocolos de validação transparentes, associados aos cenários de uso que efetivamente caracterizam. Longe de desqualificar a metodologia do artigo de referência, a avaliação desenvolvida a complementa, distinguindo o regime de otimização em *loci* conhecidos do regime de descoberta em *loci* inéditos, e reportando a métrica pertinente a cada um.

Cabe, a esse respeito, uma nota final de honestidade metodológica. A validação externa produziu um resultado predominantemente negativo, e ele é reportado como tal,

sem atenuação. Esse desfecho não é anômalo no campo, como observado no próprio artigo de referência, que submeteu o mesmo alvo ao PRIDICT 2.0 e ao DeepPrime, obtendo igualmente insucesso, e concluiu recomendando que a triagem experimental não seja integralmente substituída por predição computacional. O valor dessa validação externa não reside em confirmar o que as métricas internas já sugeriam, mas em revelar o que elas não podem revelar — que, neste caso, ela converteu uma expectativa tácita de generalidade em um domínio de aplicabilidade explícito e verificável.

6 Uso de Inteligência Artificial Generativa

A aplicação de inteligência artificial generativa ao longo do desenvolvimento desse trabalho se deu a partir do uso dos diferentes modelos pertencentes à família Claude, desenvolvidos pela Anthropic. Os usos específicos consistiram na compreensão do código originalmente legado do repositório pertencente aos autores do DTMP-Prime, às etapas de produção de *scripts* ou trechos de código ausentes no repositório legado, bem como à produção da interface gráfica e redação da documentação.

7 Agradecimentos

Gostaria de agradecer profundamente a Letícia Nunes de Souza Andrade por confiar a mim a oportunidade de trabalhar nesse projeto, bem como por toda a ajuda fornecida durante as etapas de desenvolvimento, desde os fundamentos mais basilares sobre *Prime Editing* até a contribuição com ideias de funcionalidades para a interface gráfica. Agradeço também a Joaquim Júnior Ferola Fonseca pela contribuição com os esforços iniciais na compreensão técnica do DTMP-Prime.

Ademais, agradeço a Angelica Jardim Costa e a Clarissa Brenda Alves Cavalcante pelas orientações e sugestões de ordem técnica que guiaram o desenvolvimento deste projeto de maneira a melhor se adequar às necessidades internas do Grupo de Sistemas Geneticamente Modificados do LNBio.

Por fim, agradeço a Ângela Saito por facilitar a comunicação com a Equipe de Dados Biológicos do LNBio e, conseqüentemente, a José Geraldo de Carvalho Pereira e a Rick Hideki Hokama por colaborarem no planejamento das necessidades computacionais e infraestruturais futuras do projeto, relativas à disponibilização da versão final para uso em rotina laboratorial.

Referências

- [1] Roghayyeh Alipanahi, Leila Safari, and Alireza Khanteymoori. DTMP-prime: A deep transformer-based model for predicting prime editing efficiency and PegRNA activity. *Molecular Therapy Nucleic Acids*, 35(4):102370, December 2024.
- [2] Andrew V. Anzalone, Peyton B. Randolph, Jessie R. Davis, Alexander A. Sousa, Luke W. Koblan, Jonathan M. Levy, Peter J. Chen, Christopher Wilson, Gregory A. Newby, Aditya Raguram, and David R. Liu. Search-and-replace genome editing without double-strand breaks or donor DNA. *Nature*, 576(7785):149–157, December 2019.
- [3] Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, San Francisco California USA, August 2016. ACM.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North*, pages 4171–4186, Minneapolis, Minnesota, 2019. Association for Computational Linguistics.
- [5] Yanrong Ji, Zhihan Zhou, Han Liu, and Ramana V Davuluri. DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome. *Bioinformatics*, 37(15):2112–2120, August 2021.
- [6] Melissa J Landrum, Shanmuga Chitipiralla, Garth R Brown, Chao Chen, Baoshan Gu, Jennifer Hart, Douglas Hoffman, Wonhee Jang, Kuljeet Kaur, Chunlei Liu, Vitaly Lyoshin, Zenith Maddipatla, Rama Maiti, Joseph Mitchell, Nuala OLeary, George R Riley, Wen Yao Shi, George Zhou, Valerie Schneider, Donna Maglott, J Bradley Holmes, and Brandi L Kattman. ClinVar: improvements to accessing data. *Nucleic Acids Research*, 48(D1):D835–D844, January 2020.
- [7] Nicolas Mathis, Ahmed Allam, András Tálas, Lucas Kissling, Elena Benvenuto, Lukas Schmidheini, Ruben Schep, Tanav Damodharan, Zsolt Balázs, Sharan Janjuha, Eleonora I. Ioannidi, Desirée Böck, Bas Van Steensel, Michael Krauthammer, and Gerald Schwank. Machine learning prediction of prime editing efficiency across diverse chromatin contexts. *Nature Biotechnology*, 43(5):712–719, May 2025.
- [8] Marius Mosbach, Maksym Andriushchenko, and Dietrich Klakow. On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines, 2020. Version Number: 3.

- [9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [10] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, 2019. Version Number: 4.
- [11] Alexander A. Sousa, Colin Hemez, Lei Lei, Soumba Traore, Katarina Kulhankova, Gregory A. Newby, Jordan L. Doman, Keyede Oye, Smriti Pandey, Philip H. Karp, Paul B. McCray, and David R. Liu. Systematic optimization of prime editing for the efficient functional correction of CFTR F508del in human airway epithelial cells. *Nature Biomedical Engineering*, 9(1):7–21, January 2025.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, 2017. Version Number: 7.
- [13] Goosang Yu, Hui Kwon Kim, Jinman Park, Hyunjong Kwak, Yumin Cheong, Dongyoung Kim, Jiyun Kim, Jisung Kim, and Hyongbum Henry Kim. Prediction of efficiencies for diverse prime editing systems in multiple cell types. *Cell*, 186(10):2256–2272.e23, May 2023.